Pengenalan Framework CodeIgniter 4: SIAKAD dan Toko Online

Nama: Mathias Adya Diwangkara Suganda
NRP: 5025231015
Kelas: Pemrograman Berbasis Kerangka Kerja (D)

Kode: folder `siakad/` dan `toko-online/` (CodeIgniter 4.7.6)
Source code (GitHub): github.com/MathiasAdya/pbkk-codeigniter4
Referensi: [LINK]

Soal: Latihan 1 dan Latihan 2

Latihan 1 meminta penerapan materi pengenalan CodeIgniter. Setiap topik dikerjakan di dalam aplikasi Latihan 2:

No	Topik Latihan 1	Penerapan
1	Instalasi dan konfigurasi CI	Instalasi CI 4.7.6 lewat Composer, konfigurasi `.env` (baseURL, database), migration dan seeder (Bagian 1).
2	Router dan Controller	`app/Config/Routes.php` dengan route group, namespace `Admin`, parameter `(:num)`, filter per group.
3	View Bootstrap	Halaman publik dan form login memakai Bootstrap 4, view dipecah menjadi layout dan partial.
4	SB Admin	Layout `layouts/admin.php` memakai template **SB Admin 2**: sidebar per role, topbar, card, dan tabel.
5	Model dan CRUD	Model CI4 dengan validasi, query builder, join, dan paginasi. CRUD lengkap untuk master data.
6	File upload	Upload foto mahasiswa dan gambar produk dengan validasi `is_image`, `max_size`, dan `mime_in`.
7	Hak akses user	Login dengan password ter-hash, session, dan `AuthFilter` berparameter role (`auth:admin`).

Latihan 2 meminta dua aplikasi: **SIAKAD** (Sistem Informasi Akademik) dan **Toko Online**, keduanya dengan CodeIgniter.

Pendahuluan

CodeIgniter 4 adalah framework PHP berpola **MVC** (Model-View-Controller) yang ringan dan cepat. Request masuk lewat `public/index.php` lalu dicocokkan oleh **Router**. Sebelum masuk ke Controller, request melewati **Filter** (misalnya CSRF dan cek login). Controller mengambil data lewat **Model** dan mengirimkannya ke **View**. Laporan ini membahas pembuatan dua aplikasi:

- **SIAKAD**: tiga role (admin, dosen, mahasiswa). Admin mengelola mahasiswa, dosen, dan matakuliah. Mahasiswa mengisi KRS dan melihat KHS beserta IP/IPK. Dosen menginput nilai.
- **Toko Online**: katalog produk dengan kategori, keranjang belanja, checkout, riwayat pesanan, serta panel admin untuk produk, kategori, dan pesanan.

Lingkungan pengembangan: Windows 11 + WSL, XAMPP (PHP 8.2 dan MariaDB), Composer, dan browser Microsoft Edge.

Bagian 1: Instalasi dan Konfigurasi CodeIgniter 4

1 Instalasi CodeIgniter lewat Composer

Proyek dibuat dari *app starter* resmi. Hasilnya adalah folder dengan struktur `app/` (kode aplikasi), `public/` (document root), `writable/` (cache, log, session), dan `vendor/` (framework).

```
composer create-project codeigniter4/appstarter siakad
composer create-project codeigniter4/appstarter toko-online
```

2 Konfigurasi .env

File `env` disalin menjadi `.env`, lalu baseURL, environment, dan koneksi database diatur. CSRF diaktifkan secara global di `app/Config/Filters.php` sehingga setiap form wajib membawa `csrf_field()`.

```
CI_ENVIRONMENT = development
app.baseURL = 'http://localhost:8080/'

database.default.hostname = localhost
database.default.database = ci_siakad
database.default.username = root
database.default.password =
database.default.DBDriver = MySQLi
database.default.port = 3306
```

3 Membuat Database dengan Migration dan Seeder

Skema tabel ditulis sebagai **migration** agar bisa dibuat ulang di komputer mana pun. Data awal (akun admin, dosen, mahasiswa, dan matakuliah) diisi lewat **seeder**.

```
CREATE DATABASE ci_siakad;
php spark migrate
php spark db:seed SiakadSeeder
```

4 Running Aplikasi

Di XAMPP ekstensi `intl` (wajib untuk CI4) tidak aktif secara default. Perintah `php spark serve` menjalankan proses PHP baru tanpa opsi tambahan, jadi server dijalankan dengan `serve.bat` yang memanggil built-in server PHP secara langsung:

```
@echo off
rem Jalankan dev server dengan ekstensi intl
cd /d %~dp0
C:\xampp\php\php.exe -d extension=intl -S localhost:%1 -t public vendor\codeigniter4\framework\system\rewrite.php
```

```
siakad\serve.bat 8080        -> http://localhost:8080
toko-online\serve.bat 8081   -> http://localhost:8081
```

Seluruh source code tersimpan di repository GitHub MathiasAdya/pbkk-codeigniter4 (publik). Folder `vendor/` dan file `.env` tidak ikut di-commit, sehingga setelah clone perlu dijalankan:

```
git clone https://github.com/MathiasAdya/pbkk-codeigniter4.git
cd pbkk-codeigniter4/siakad      # atau toko-online
composer install
copy env .env                    # atur baseURL dan database
php spark migrate
php spark db:seed SiakadSeeder   # toko-online: TokoSeeder
```

Akun untuk mencoba aplikasi:

Aplikasi	Role	Username / Password
SIAKAD	Admin	`admin` / `admin123`
SIAKAD	Dosen	NIDN / NIDN, contoh `0011223301`
SIAKAD	Mahasiswa	NRP / NRP, contoh `5025231015`
Toko Online	Admin	`admin` / `admin123`

Toko Online	Customer	`budi` / `budi123` (atau daftar akun baru di `/register`)

5 Kendala dan Solusi

Kendala	Penyebab	Solusi
Error *"The framework needs the following extension(s) installed and loaded: intl"*	Ekstensi intl tidak aktif di php.ini bawaan XAMPP.	Menjalankan PHP dengan `-d extension=intl`.
`php spark serve` tetap gagal	spark menjalankan proses PHP baru tanpa opsi `-d`.	Membuat `serve.bat` yang menjalankan `php -S` langsung dengan `rewrite.php`.
Semua relasi ikut terhapus (CASCADE), padahal seharusnya RESTRICT atau SET NULL	Urutan argumen `addForeignKey()` tertukar.	Urutan yang benar adalah `addForeignKey($field, $table, $col, $onUpdate, $onDelete)`. onUpdate ditulis lebih dulu.
Error 500 *"Call to undefined function user()"* di halaman KRS/Nilai	Helper dimuat di `initController()`, yang dipanggil setelah `__construct()`.	Helper `siakad` di-autoload lewat `Config/Autoload::$helpers`.
Validasi gambar gagal di server tanpa GD	Ekstensi GD tidak tersedia.	Validasi cukup memakai `is_image`, `mime_in`, dan `max_size`, tanpa resize gambar.
Error 500 saat menghapus kategori yang masih memiliki produk	FK `RESTRICT` membuat MariaDB menolak DELETE dan CI4 melempar `DatabaseException`.	Exception ditangkap di controller dan diganti pesan *"Kategori tidak dapat dihapus karena masih memiliki produk."*
Total pendapatan di dashboard selalu kosong	`selectSum()` pada Model bercampur dengan select bawaan model.	Query dijalankan lewat `$model->builder()->selectSum('total')->get()->getRowArray()`.
Stok bisa minus jika dua pelanggan checkout bersamaan	Stok dicek lalu dikurangi di dua query terpisah tanpa penguncian.	Checkout dijalankan dalam transaksi dengan `SELECT ... FOR UPDATE`; jumlah di keranjang juga dibatasi stok.

Bagian 2: Aplikasi SIAKAD

1 Merancang Database

Akun login semua role disimpan di tabel `users`, sedangkan data profil disimpan di tabel `dosen` dan `mahasiswa` yang terhubung lewat `user_id`. Tabel `krs` menyimpan matakuliah yang diambil mahasiswa per tahun ajaran beserta nilainya.

```
users 1 ── 1 dosen        (user_id, ON DELETE CASCADE)
users 1 ── 1 mahasiswa    (user_id, ON DELETE CASCADE)
dosen 1 ── N mahasiswa    (dosen_wali_id, ON DELETE SET NULL)
dosen 1 ── N matakuliah   (dosen_id, ON DELETE SET NULL)
mahasiswa 1 ── N krs N ── 1 matakuliah
              (CASCADE)        (RESTRICT: matakuliah yang sudah diambil tidak bisa dihapus)
```

```
// app/Database/Migrations/2026-11-04-000001_CreateSiakadTables.php
$this->forge->addForeignKey('mahasiswa_id', 'mahasiswa', 'id', 'CASCADE', 'CASCADE');
$this->forge->addForeignKey('matakuliah_id', 'matakuliah', 'id', 'CASCADE', 'RESTRICT');
$this->forge->addUniqueKey(['mahasiswa_id', 'matakuliah_id', 'tahun_ajaran']);
$this->forge->createTable('krs');
```

2 Router dan Controller

Route dikelompokkan per role. Setiap group diberi filter, sehingga controller tidak perlu memeriksa hak akses sendiri. Route CRUD admin dibuat dengan perulangan agar tiga master data memakai pola URL yang sama.

```
$routes->get('/', 'Home::index');
$routes->get('login', 'Auth::login');
$routes->post('login', 'Auth::attempt');
$routes->get('logout', 'Auth::logout');
$routes->get('dashboard', 'Dashboard::index', ['filter' => 'auth']);

// Admin: CRUD master data
$routes->group('admin', ['namespace' => 'App\Controllers\Admin', 'filter' => 'auth:admin'], static function ($routes) {
    foreach (['mahasiswa' => 'Mahasiswa', 'dosen' => 'Dosen', 'matakuliah' => 'Matakuliah'] as $uri => $controller) {
        $routes->get($uri, "{$controller}::index");
        $routes->get("{$uri}/new", "{$controller}::new");
        $routes->post($uri, "{$controller}::create");
        $routes->get("{$uri}/(:num)/edit", "{$controller}::edit/$1");
        $routes->post("{$uri}/(:num)", "{$controller}::update/$1");
        $routes->post("{$uri}/(:num)/delete", "{$controller}::delete/$1");
    }
});

// Mahasiswa: KRS & KHS
$routes->group('', ['filter' => 'auth:mahasiswa'], static function ($routes) {
    $routes->get('krs', 'Krs::index');
    $routes->post('krs', 'Krs::ambil');
    $routes->post('krs/(:num)/delete', 'Krs::drop/$1');
    $routes->get('khs', 'Krs::khs');
});

// Dosen: input nilai
$routes->group('nilai', ['filter' => 'auth:dosen'], static function ($routes) {
    $routes->get('/', 'Nilai::index');
    $routes->get('(:num)', 'Nilai::show/$1');
    $routes->post('(:num)', 'Nilai::save/$1');
});
```

Aksi hapus dan ubah memakai POST, bukan GET, sehingga selalu dilindungi token CSRF.

3 View Bootstrap dan Template SB Admin 2

Halaman beranda dan login memakai Bootstrap. Halaman setelah login memakai layout `layouts/admin.php` berbasis **SB Admin 2** (CSS/JS dari CDN jsDelivr). View lain cukup meng-*extend* layout tersebut dan mengisi section `content`. Menu sidebar (`layouts/_sidebar.php`) ditampilkan sesuai role yang login, dan pesan flashdata ditampilkan oleh `layouts/_alerts.php`.

```
&lt;!-- app/Views/admin/mahasiswa/index.php --&gt;
&lt;?= $this-&gt;extend('layouts/admin') ?&gt;
&lt;?= $this-&gt;section('content') ?&gt;
  ... tabel data, form pencarian, $pager-&gt;links() ...
&lt;?= $this-&gt;endSection() ?&gt;
```

4 Model dan CRUD

Setiap tabel punya Model dengan `$allowedFields` (melindungi dari mass assignment), `$validationRules`, dan timestamp otomatis. Password di-hash otomatis lewat callback `beforeInsert` dan `beforeUpdate`:

```
class UserModel extends Model
{
    protected $table         = 'users';
    protected $allowedFields = ['username', 'password', 'nama', 'role'];
    protected $beforeInsert  = ['hashPassword'];
    protected $beforeUpdate  = ['hashPassword'];

    protected function hashPassword(array $data): array
    {
        if (! empty($data['data']['password'])) {
            $data['data']['password'] = password_hash($data['data']['password'], PASSWORD_DEFAULT);
        } else {
            unset($data['data']['password']);
        }
        return $data;
    }

    public function attempt(string $username, string $password): ?array
    {
        $user = $this->where('username', $username)->first();
        return ($user && password_verify($password, $user['password'])) ? $user : null;
    }
}
```

Saat admin menambah dosen atau mahasiswa, akun login di `users` dan profilnya dibuat dalam satu transaksi database (trait `HasUserAccount`). Username dan password awal sama dengan NIDN/NRP. Halaman daftar dilengkapi pencarian dan paginasi (`paginate(10)`). Matakuliah yang sudah ada di KRS tidak bisa dihapus karena FK RESTRICT, dan admin mendapat pesan error yang jelas.

5 Upload Foto Mahasiswa

Form mahasiswa memakai `form_open_multipart()`. File divalidasi (gambar, maks 1 MB, JPEG/PNG/WEBP), lalu disimpan dengan nama acak ke `public/uploads/foto/`. Foto lama dihapus saat diganti atau saat data dihapus.

```
private const FOTO_RULE = 'is_image[foto]|max_size[foto,1024]|mime_in[foto,image/jpeg,image/png,image/webp]';

public function create()
{
    $data = $this->input();
    if (! $this->model->validate($data) || ! $this->validate(['foto' => 'uploaded[foto]|' . self::FOTO_RULE])) {
        return $this->back();
    }
    $data['foto'] = $this->storeFoto($this->request->getFile('foto'));
    ...
}

private function storeFoto(UploadedFile $file): string
{
    $name = $file->getRandomName();
    $file->move(self::FOTO_DIR, $name);
    return $name;
}
```

6 Hak Akses User dengan Filter

Login menyimpan `logged_in`, `role`, dan data user ke session, setelah ID session diganti (`session()->regenerate()`) untuk mencegah session fixation. `AuthFilter` menerima argumen role dari route, misalnya `auth:admin`:

```
class AuthFilter implements FilterInterface
{
    public function before(RequestInterface $request, $arguments = null)
    {
        $session = session();

        if (! $session->get('logged_in')) {
            return redirect()->to('/login')->with('error', 'Silakan login terlebih dahulu.');
        }

        if (! empty($arguments) && ! in_array($session->get('role'), $arguments, true)) {
            return redirect()->to('/dashboard')->with('error', 'Anda tidak memiliki akses ke halaman tersebut.');
        }
    }
}

// app/Config/Filters.php
public array $aliases = [
    ...
    'auth' => \App\Filters\AuthFilter::class,
];
```

7 KRS dan KHS Mahasiswa

Mahasiswa memilih matakuliah untuk semester berjalan (`2026/2027 Ganjil`). Model menolak matakuliah yang sudah diambil dan total SKS yang melebihi 24. Pembatalan hanya berlaku untuk KRS milik sendiri, pada semester berjalan, yang belum dinilai. Syarat ini ditulis langsung di query DELETE, sehingga mahasiswa lain tidak bisa membatalkan KRS orang lain dengan mengganti ID.

```
public function ambil(int $mahasiswaId, array $matakuliah): ?string
{
    $krs = $this->forMahasiswa($mahasiswaId);

    if (in_array($matakuliah['id'], array_column($krs, 'matakuliah_id'))) {
        return 'Matakuliah sudah ada di KRS.';
    }
    if (array_sum(array_column($krs, 'sks')) + $matakuliah['sks'] > self::MAX_SKS) {
        return 'Total SKS melebihi batas ' . self::MAX_SKS . ' SKS.';
    }
```

```
    $this->insert([...]);
    return null;
}

// Controller Krs::drop
$this->krs->where([
    'id'           => $id,
    'mahasiswa_id' => $this->mahasiswa['id'],
    'tahun_ajaran' => KrsModel::TAHUN_AJARAN,
])->where('nilai', null)->delete();
```

KHS mengelompokkan nilai per tahun ajaran. IP dan IPK dihitung oleh helper `hitung_ip()` dengan bobot A=4, AB=3,5, B=3, BC=2,5, C=2, D=1, E=0. Matakuliah yang belum dinilai tidak ikut dihitung.

8 Input Nilai oleh Dosen

Dosen hanya melihat matakuliah yang diampunya. Membuka matakuliah dosen lain (misalnya `/nilai/2`) menghasilkan 404. Nilai seluruh peserta disimpan sekaligus dari satu form, dan setiap nilai divalidasi terhadap daftar huruf yang sah.

Bagian 3: Aplikasi Toko Online

1 Merancang Database

Akun admin dan pelanggan disimpan di tabel `users` dengan kolom `role`. Setiap pesanan (`orders`) memiliki banyak `order_items`, dan setiap item menyimpan **harga saat dibeli** (snapshot) sehingga perubahan harga produk tidak mengubah pesanan lama.

```
kategori 1 ── N produk        (kategori_id, RESTRICT: kategori berisi produk tidak bisa dihapus)
users    1 ── N orders        (user_id, CASCADE)
orders   1 ── N order_items   (order_id, CASCADE)
produk   1 ── N order_items   (produk_id, RESTRICT: produk yang pernah dipesan tidak bisa dihapus)
```

```
// app/Database/Migrations/2026-11-04-000001_CreateTokoTables.php
// addForeignKey(field, table, column, onUpdate, onDelete)
$this->forge->addForeignKey('kategori_id', 'kategori', 'id', 'CASCADE', 'RESTRICT');
$this->forge->createTable('produk');
...
$this->forge->addForeignKey('order_id', 'orders', 'id', 'CASCADE', 'CASCADE');
$this->forge->addForeignKey('produk_id', 'produk', 'id', 'CASCADE', 'RESTRICT');
$this->forge->createTable('order_items');
```

2 Router dan Filter Hak Akses

Etalase dan keranjang bersifat publik. Checkout dan riwayat pesanan hanya untuk role `customer`, sedangkan panel admin memakai filter `auth:admin`. Filter yang sama (`AuthFilter`) dipakai seperti di SIAKAD: belum login diarahkan ke `/login`, role yang salah diarahkan ke beranda dengan pesan *"Anda tidak memiliki akses ke halaman tersebut."*

```
$routes->get('/', 'Toko::index');
$routes->get('produk/(:num)', 'Toko::detail/$1');

// Keranjang (session, boleh tanpa login)
$routes->get('keranjang', 'Keranjang::index');
$routes->post('keranjang/tambah', 'Keranjang::tambah');
$routes->post('keranjang/update', 'Keranjang::update');
$routes->post('keranjang/hapus/(:num)', 'Keranjang::hapus/$1');

// Customer: checkout & pesanan
$routes->group('', ['filter' => 'auth:customer'], static function ($routes) {
    $routes->get('checkout', 'Checkout::index');
    $routes->post('checkout', 'Checkout::proses');
    $routes->get('pesanan', 'Pesanan::index');
    $routes->get('pesanan/(:num)', 'Pesanan::show/$1');
    $routes->post('pesanan/(:num)/batal', 'Pesanan::batal/$1');
});

// Admin: dashboard, CRUD kategori & produk, kelola pesanan
$routes->group('admin', ['namespace' => 'App\Controllers\Admin', 'filter' => 'auth:admin'], static function ($routes) {
    $routes->get('/', 'Dashboard::index');
    foreach (['kategori' => 'Kategori', 'produk' => 'Produk'] as $uri => $controller) {
        // index, new, create, edit, update, delete (pola sama dengan SIAKAD)
    }
    $routes->get('pesanan', 'Pesanan::index');
    $routes->get('pesanan/(:num)', 'Pesanan::show/$1');
```

```
    $routes->post('pesanan/(:num)/status', 'Pesanan::status/$1');
});
```

3 Katalog Produk

Beranda menampilkan produk dalam kartu Bootstrap dengan filter kategori, pencarian nama, dan paginasi 9 produk per halaman. Query dibangun di `ProdukModel::daftar()` sehingga dipakai juga oleh halaman produk admin.

```
public function index(): string
{
    $q        = trim((string) $this->request->getGet('q'));
    $kategori = (int) $this->request->getGet('kategori');
    $model    = model(ProdukModel::class);

    return view('toko/index', [
        'produk'   => $model->daftar($q, $kategori)->paginate(9),
        'pager'    => $model->pager,
        'kategori' => model(KategoriModel::class)->orderBy('nama')->findAll(),
        ...
    ]);
}
```

4 Keranjang Belanja (Session)

Keranjang disimpan di session dalam bentuk `[produk_id => qty]` melalui library `App\Libraries\Cart`, sehingga pengunjung bisa berbelanja sebelum login. Jumlah selalu dibatasi stok yang tersedia.

```
public function add(int $id, int $qty): ?string
{
    $produk = model(ProdukModel::class)->find($id);
    if (! $produk) {
        return 'Produk tidak ditemukan.';
    }
    if ($produk['stok'] < 1) {
        return 'Stok produk habis.';
    }

    $cart      = $this->raw();
    $baru      = ($cart[$id] ?? 0) + max(1, $qty);
    $cart[$id] = min($baru, (int) $produk['stok']);
    session()->set(self::KEY, $cart);

    return $baru > $produk['stok'] ? 'Jumlah dibatasi sesuai stok tersedia (' . $produk['stok'] . ').' : null;
}
```

5 Checkout dalam Satu Transaksi

Stok bisa berubah antara saat produk dimasukkan ke keranjang dan saat checkout, misalnya karena pelanggan lain membeli produk yang sama. Karena itu `OrderModel::buat()` mengunci baris produk dengan `SELECT ... FOR UPDATE`, memeriksa stok, menguranginya, lalu menyimpan order dan item dalam satu transaksi. Bila satu item gagal, semua perubahan di-rollback.

```
$db->transBegin();
try {
    foreach ($items as $item) {
        $p = $db->query('SELECT id, nama, harga, stok FROM produk WHERE id = ? FOR UPDATE', [$item['id']])->getRowArray();
        if ($p['stok'] < $item['qty']) {
            throw new RuntimeException("Stok \"{$p['nama']}\" tidak mencukupi (tersisa {$p['stok']}).");
        }
        $total  += $p['harga'] * $item['qty'];
        $baris[] = ['produk_id' => $p['id'], 'qty' => $item['qty'], 'harga' => $p['harga']];
        $db->query('UPDATE produk SET stok = stok - ? WHERE id = ?', [$item['qty'], $p['id']]);
    }
    $orderId = $this->insert(['user_id' => $userId, 'total' => $total, 'alamat' => $alamat, 'status' => 'pending'], true);
    foreach ($baris as $b) {
        model(OrderItemModel::class)->insert($b + ['order_id' => $orderId]);
    }
    $db->transCommit();
} catch (\Throwable $e) {
    $db->transRollback();
    throw $e;
}
```

Controller `Checkout::proses()` memvalidasi alamat, memanggil `buat()`, mengosongkan keranjang, lalu mengarahkan ke detail pesanan dengan pesan *"Pesanan berhasil dibuat."*

6 Riwayat dan Pembatalan Pesanan

Pelanggan hanya melihat pesanan miliknya (`withPelanggan($userId)`). Pesanan berstatus `pending` dapat dibatalkan, dan pembatalan mengembalikan stok produk di dalam transaksi yang sama.

```
if ($status === 'batal') {
    foreach (model(OrderItemModel::class)->where('order_id', $id)->findAll() as $it) {
        $db->query('UPDATE produk SET stok = stok + ? WHERE id = ?', [$it['qty'], $it['produk_id']]);
    }
}
$this->update($id, ['status' => $status]);
```

7 Panel Admin: CRUD Kategori dan Produk dengan Upload Gambar

Dashboard admin menampilkan jumlah produk, pesanan, pelanggan, dan total pendapatan (pesanan berstatus dibayar, dikirim, atau selesai). CRUD produk menerima gambar JPG/PNG/WEBP maksimal 2 MB yang disimpan ke `public/uploads/produk` dengan nama acak. Gambar lama dihapus saat diganti. Penghapusan yang melanggar FK RESTRICT ditangkap dan ditampilkan sebagai pesan.

```
private const GAMBAR_RULE = 'is_image[gambar]|mime_in[gambar,image/jpeg,image/png,image/webp]|max_size[gambar,2048]';

public function delete(int $id)
{
    $row = $this->findOr404($id);
    try {
        $this->model->delete($id);
    } catch (DatabaseException) {
        // FK order_items -> produk bersifat RESTRICT
        return redirect()->to('/admin/produk')->with('error', 'Produk tidak dapat dihapus karena sudah pernah dipesan.');
    }
    $this->hapusFile($row['gambar']);
    return redirect()->to('/admin/produk')->with('success', 'Produk dihapus.');
}
```

8 Pengelolaan Status Pesanan

Admin mengubah status pesanan melalui alur `pending → dibayar → dikirim → selesai`, atau `batal`. Status `batal` bersifat final: pesanan yang sudah dibatalkan tidak bisa diubah lagi, sehingga stok tidak dikembalikan dua kali.

Pembagian Tugas Kelompok

Tugas ini dikerjakan secara individu, dengan pembagian pekerjaan sebagai berikut:

Anggota	Area	Pekerjaan
Mathias (5025231015)	SIAKAD	Instalasi CI4, desain database dan migration, autentikasi dan filter role, CRUD master data, upload foto, KRS/KHS, input nilai, pengujian.
Mathias (5025231015)	Toko Online	Desain database, katalog dan kategori, keranjang, checkout, riwayat pesanan, panel admin, upload gambar produk, pengujian.
Mathias (5025231015)	Dokumentasi	Screenshot hasil dan penulisan laporan.

Alur Data

Sebagai contoh, berikut alur data saat mahasiswa mengambil matakuliah di halaman KRS:

```
Mahasiswa klik "Ambil" di krs/index.php (form POST + csrf_field)
      │  POST /krs  matakuliah_id=4
      ▼
```

```
Filter global CSRF  ──(token salah)──► error 403
       │
Filter auth:mahasiswa (AuthFilter)
       │  belum login ─► /login      role lain ─► /dashboard + pesan "tidak memiliki akses"
       ▼
Router: Krs::ambil   (konstruktor: user('id') ─► MahasiswaModel::findByUser)
       │  MatakuliahModel::find(4)
       ▼
KrsModel::ambil(mahasiswaId, matakuliah)
       │  cek duplikat ─► "Matakuliah sudah ada di KRS."
       │  cek total SKS > 24 ─► "Total SKS melebihi batas"
       │  INSERT krs (tahun_ajaran = 2026/2027 Ganjil)
       ▼
redirect('/krs')->with('success', ...)  ─► flashdata ditampilkan layouts/_alerts.php
```

Tanggung jawab tiap file

File	Tanggung jawab
`app/Config/Routes.php`	Memetakan URL ke controller dan memasang filter per group role.
`app/Config/Filters.php`	Mendaftarkan alias `auth` dan mengaktifkan CSRF global.
`app/Config/Autoload.php`	Autoload helper `siakad` (`user()`, `hitung_ip()`).
`app/Filters/AuthFilter.php`	Cek login dan role sebelum controller dijalankan.
`app/Controllers/Auth.php`	Login, logout, dan pengisian session.
`app/Controllers/Admin/*.php`	CRUD mahasiswa (dengan upload foto), dosen, dan matakuliah.
`app/Controllers/Krs.php`	KRS (ambil/batal) dan KHS mahasiswa.
`app/Controllers/Nilai.php`	Daftar matakuliah dosen dan input nilai peserta.
`app/Models/*.php`	Akses tabel, validasi, join, dan aturan bisnis (KRS, hash password, akun + profil dalam satu transaksi).
`app/Database/Migrations, Seeds`	Skema tabel dengan FK, serta data awal.
`app/Views/layouts/*.php`	Layout SB Admin 2, sidebar per role, dan alert flashdata.
`toko-online/app/Libraries/Cart.php`	Keranjang berbasis session, jumlah dibatasi stok.
`toko-online/app/Models/OrderModel.php`	Checkout dalam transaksi (FOR UPDATE), perubahan status, dan pengembalian stok saat batal.
`toko-online/app/Controllers/Keranjang.php, Checkout.php, Pesanan.php`	Alur belanja pelanggan: keranjang, checkout, riwayat dan pembatalan pesanan.
`toko-online/app/Controllers/Admin/*.php`	Dashboard, CRUD kategori dan produk (upload gambar), serta pengelolaan status pesanan.

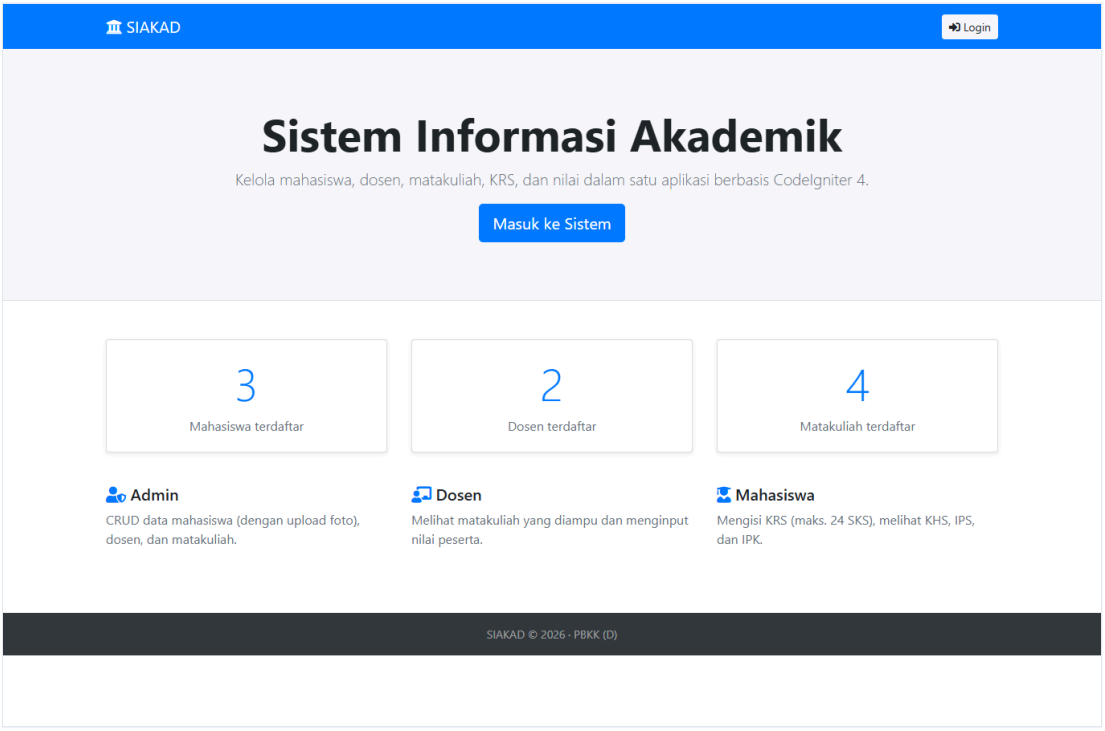
Kesimpulan

- CodeIgniter 4 memisahkan tanggung jawab dengan jelas: **Router** memetakan URL, **Filter** menangani login, role, dan CSRF, **Controller** mengatur alur, **Model** menyimpan aturan data, dan **View** hanya menampilkan.
- Hak akses cukup ditulis sekali di `AuthFilter` dan dipasang per route group, sehingga tidak ada pengecekan role yang tersebar di controller.
- Migration, seeder, dan foreign key (CASCADE, SET NULL, RESTRICT) membuat database mudah dibuat ulang sekaligus menjaga integritas data, misalnya matakuliah yang sudah diambil tidak bisa dihapus.
- Fitur bawaan seperti validasi, `UploadedFile`, paginasi, flashdata, dan query builder membuat CRUD dan upload file cepat dibuat sekaligus aman.

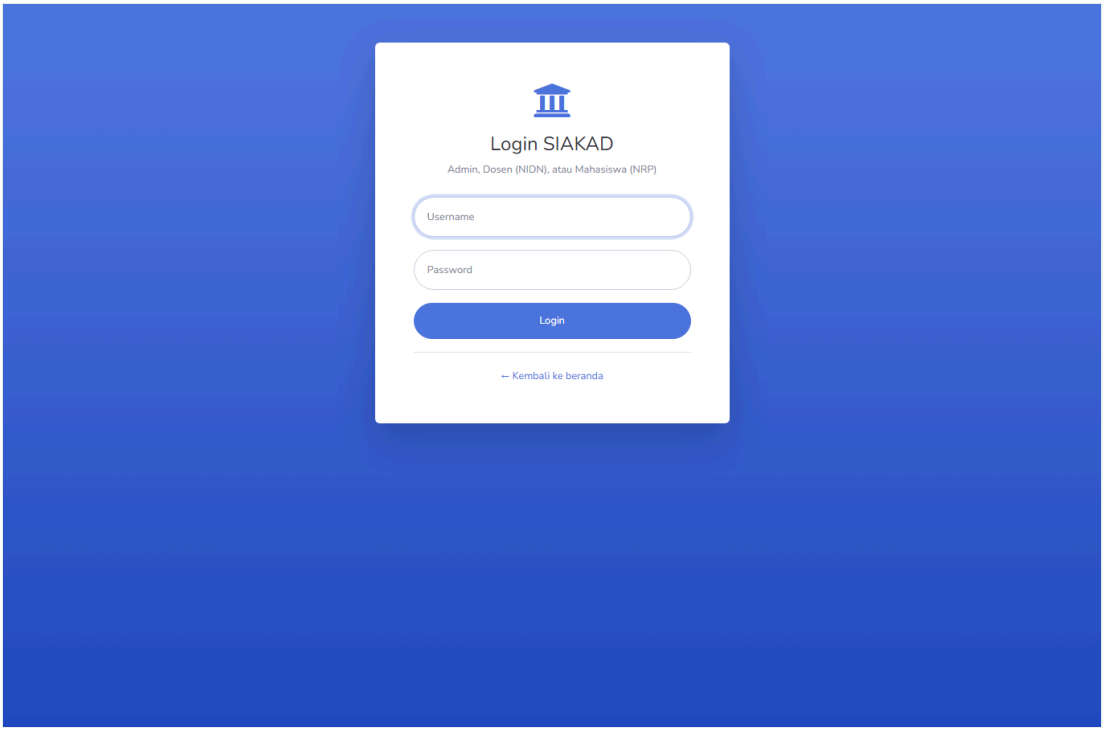
- Kedua aplikasi (SIAKAD dan Toko Online) sudah diuji untuk setiap role, termasuk kasus gagal: login salah, akses ditolak, KRS duplikat, pembatalan KRS milik orang lain, stok tidak mencukupi, dan penghapusan data yang masih dipakai.
- Untuk data yang diperebutkan seperti stok, transaksi database dengan `SELECT ... FOR UPDATE` menjamin stok tidak pernah minus dan pesanan tersimpan utuh atau tidak sama sekali.

Hasil (Foto)

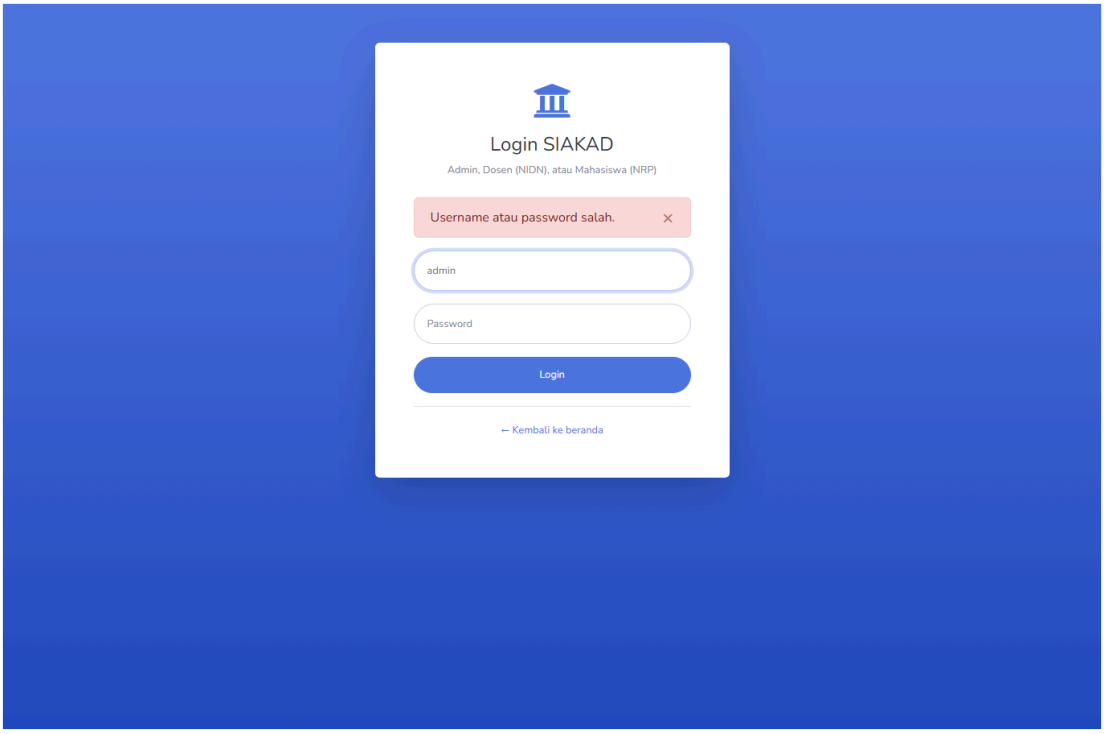
SIAKAD



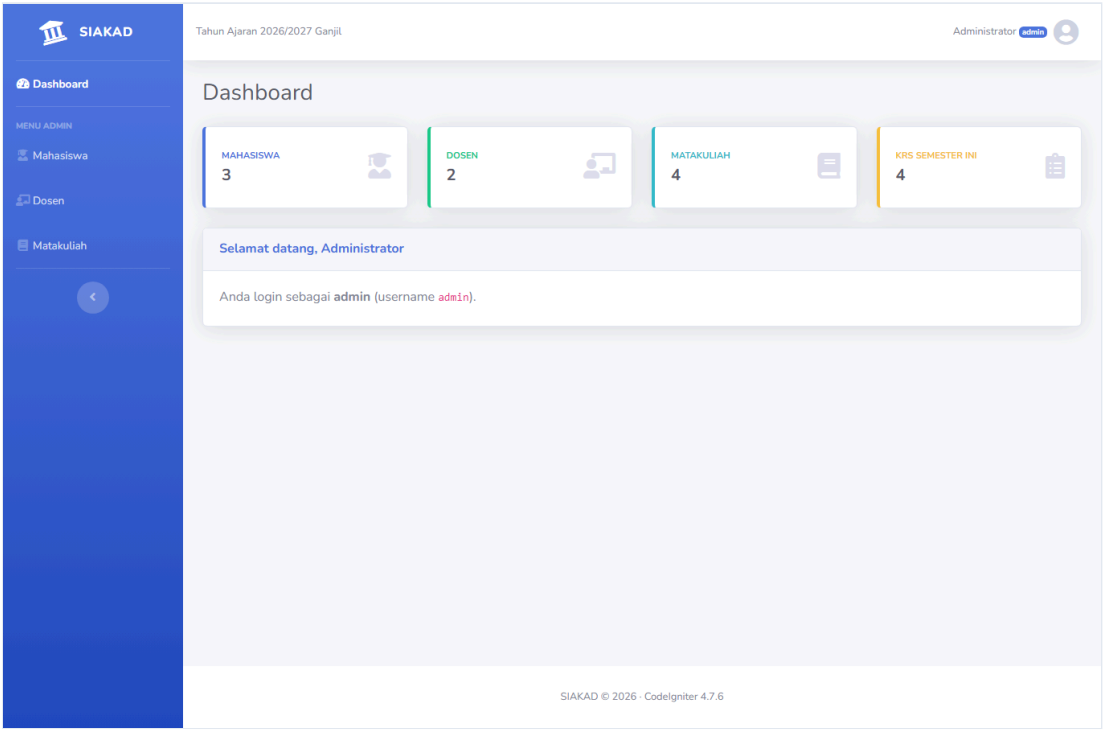
Gambar 1. Halaman beranda SIAKAD



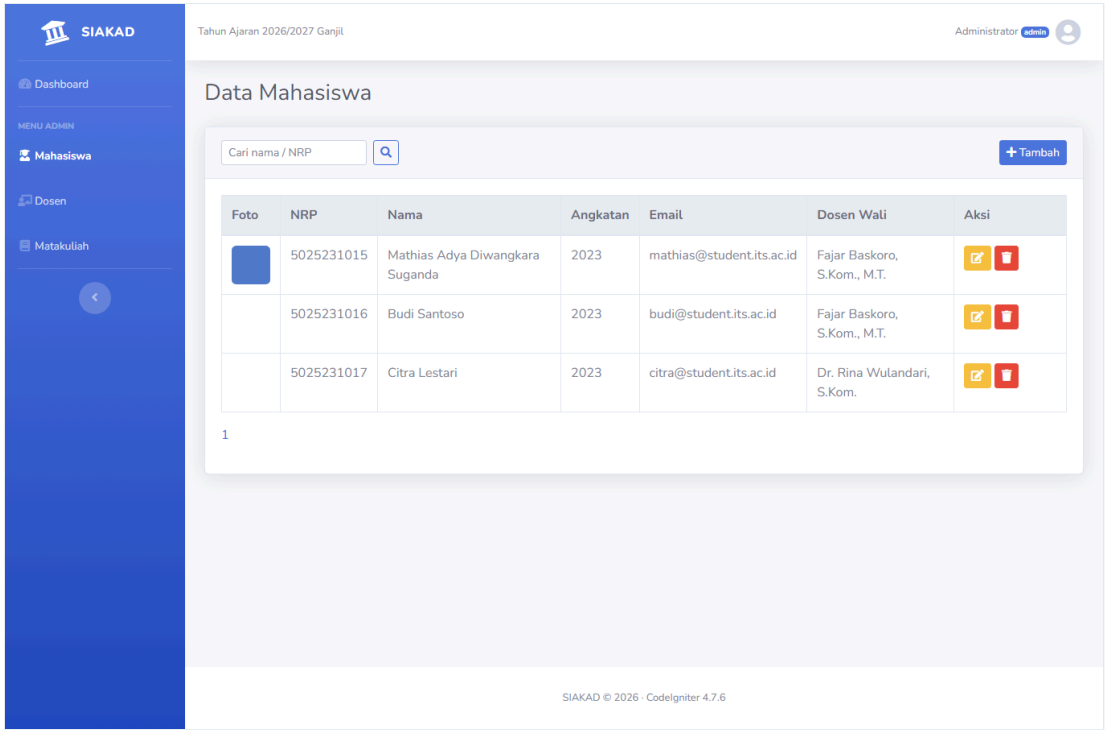
Gambar 2. Halaman login



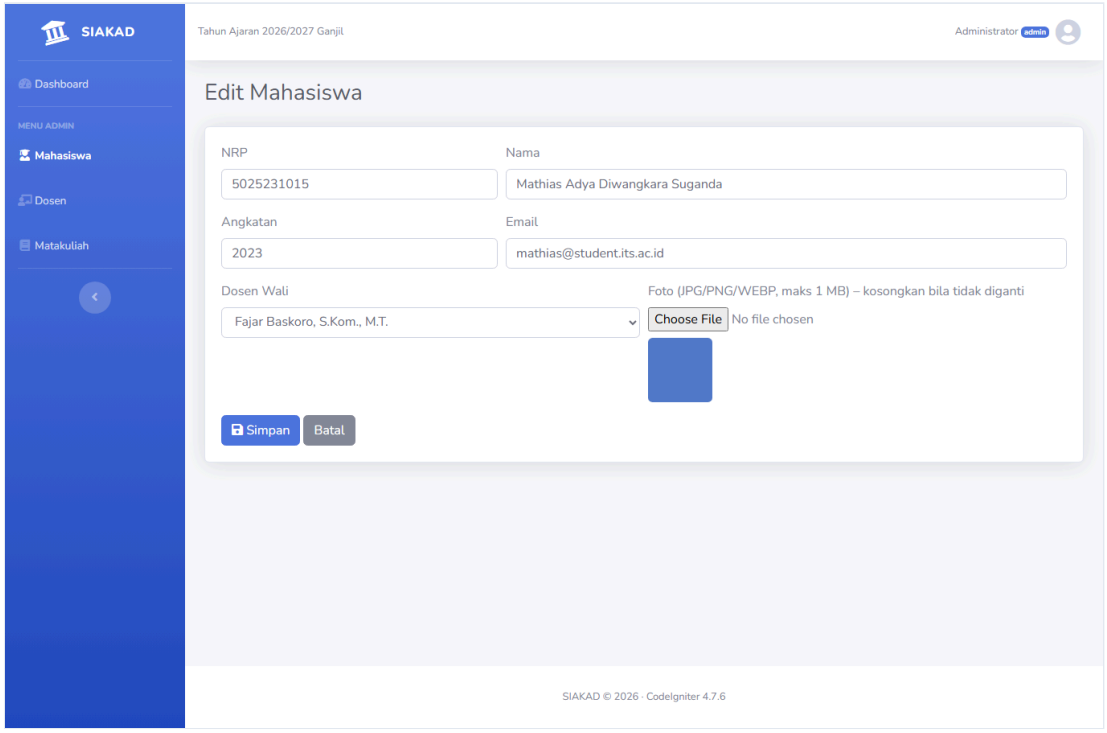
Gambar 3. Login gagal (username/password salah)



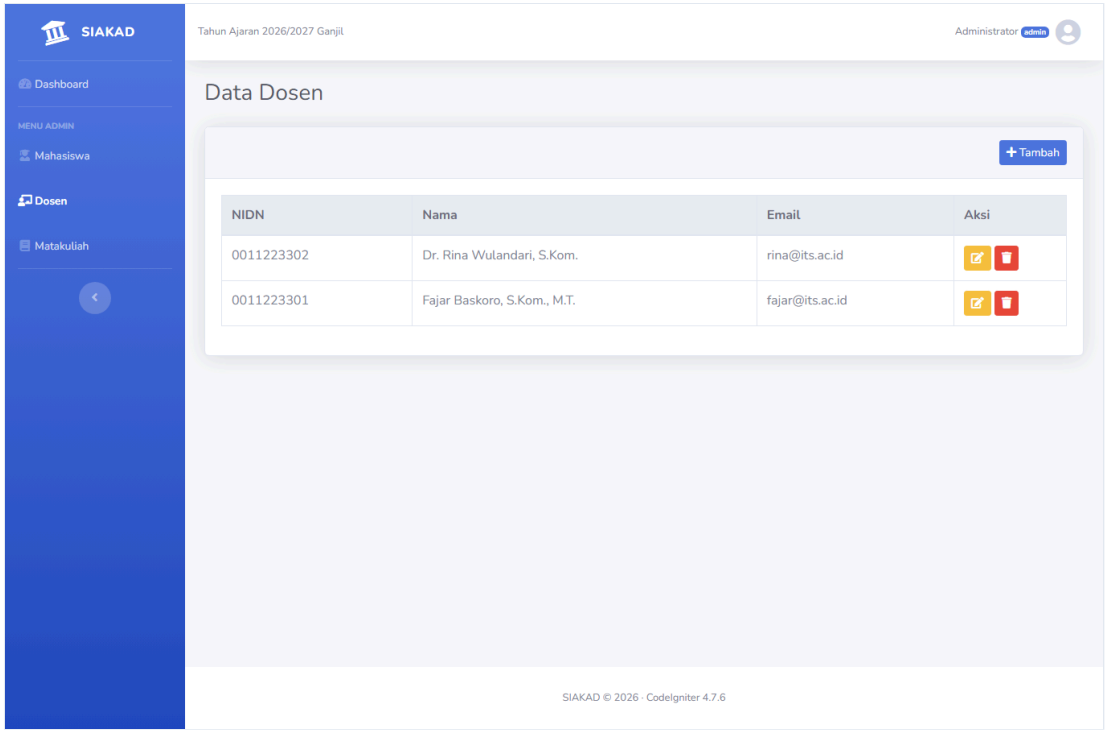
Gambar 4. Dashboard admin (SB Admin 2)



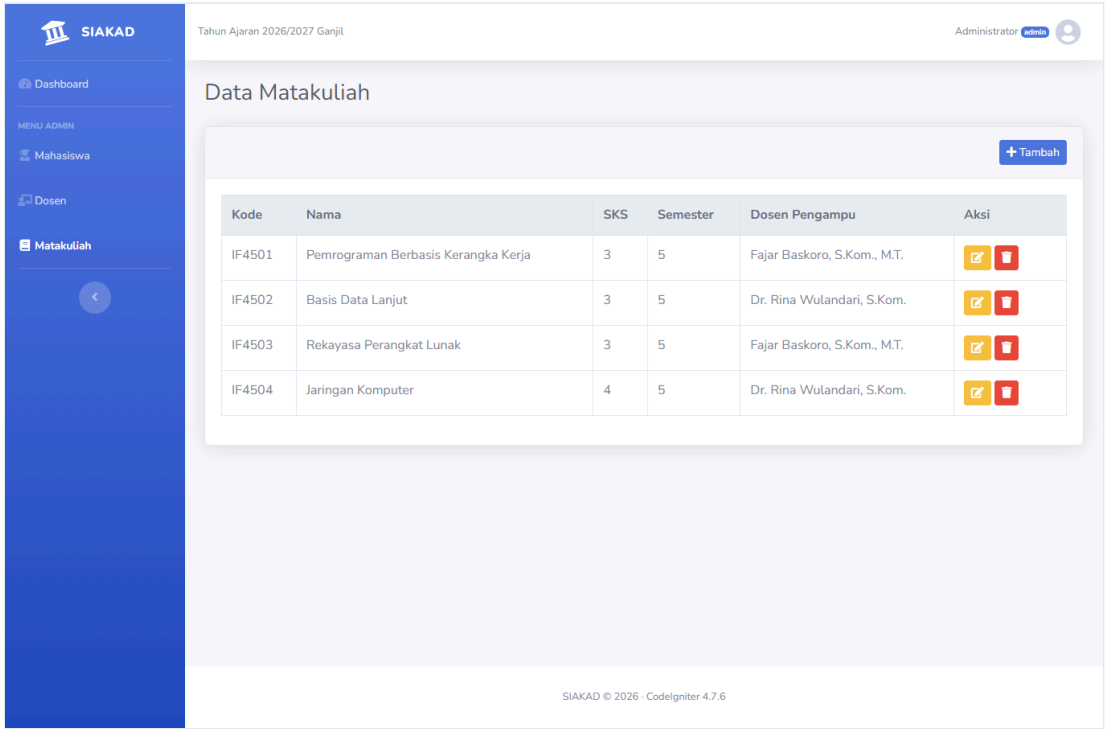
Gambar 5. Data mahasiswa dengan foto, pencarian, dan paginasi



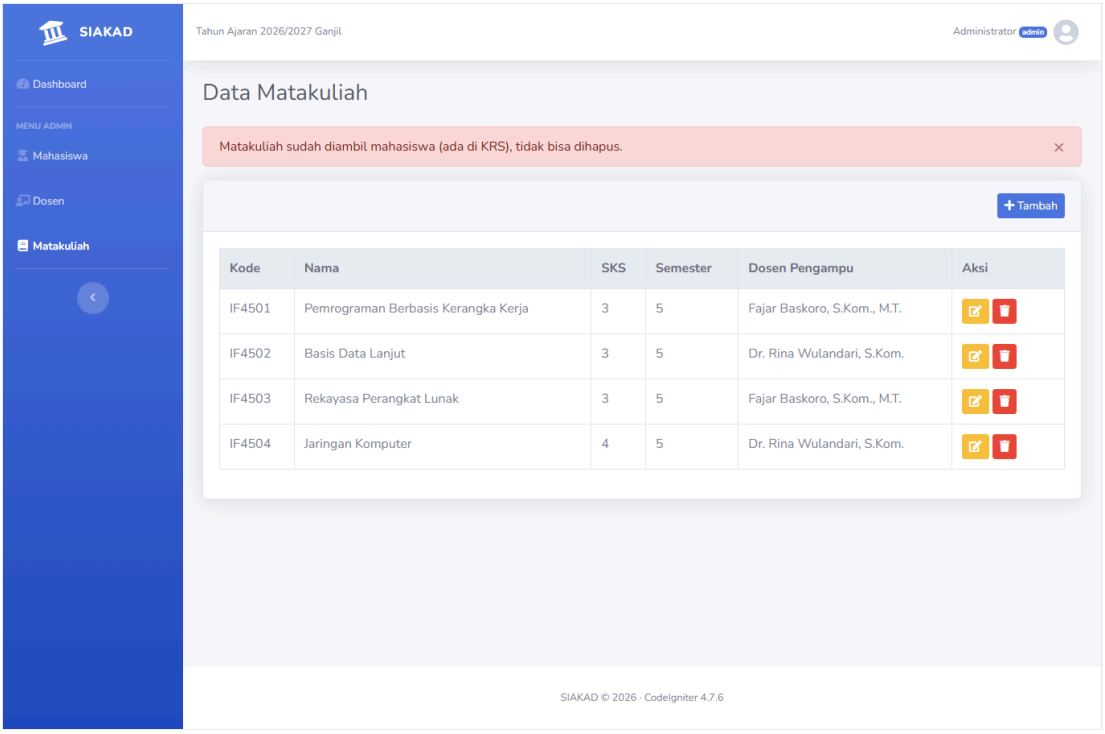
Gambar 6. Form edit mahasiswa dengan upload foto



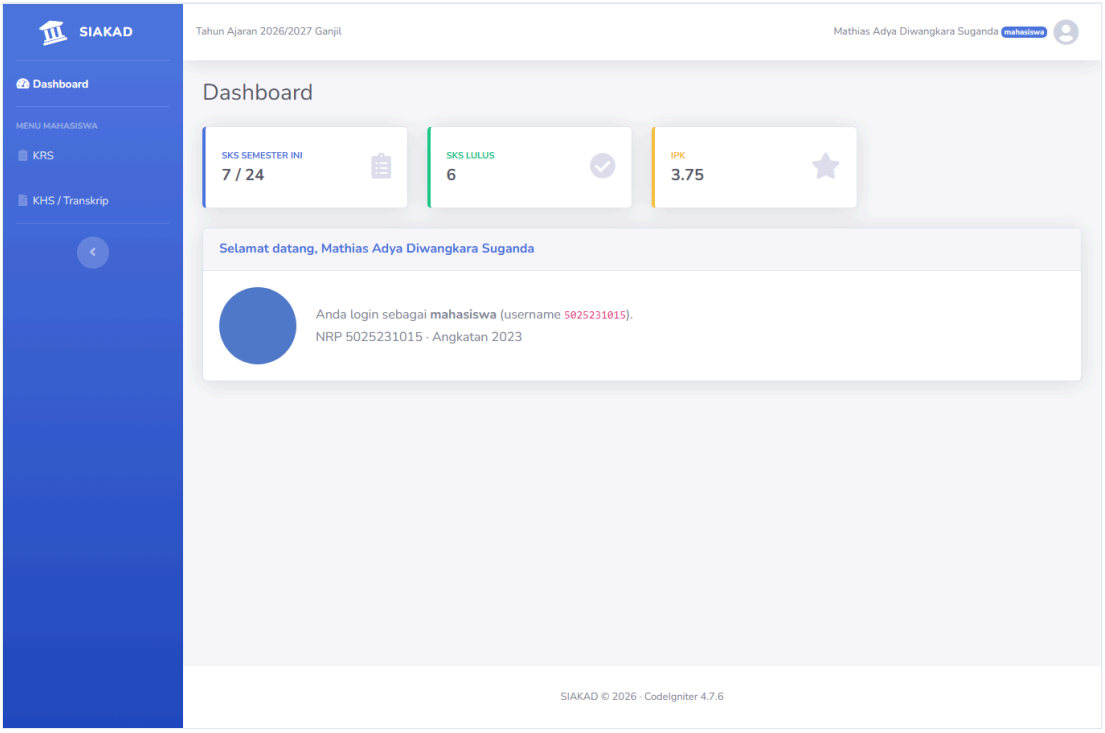
Gambar 7. Data dosen



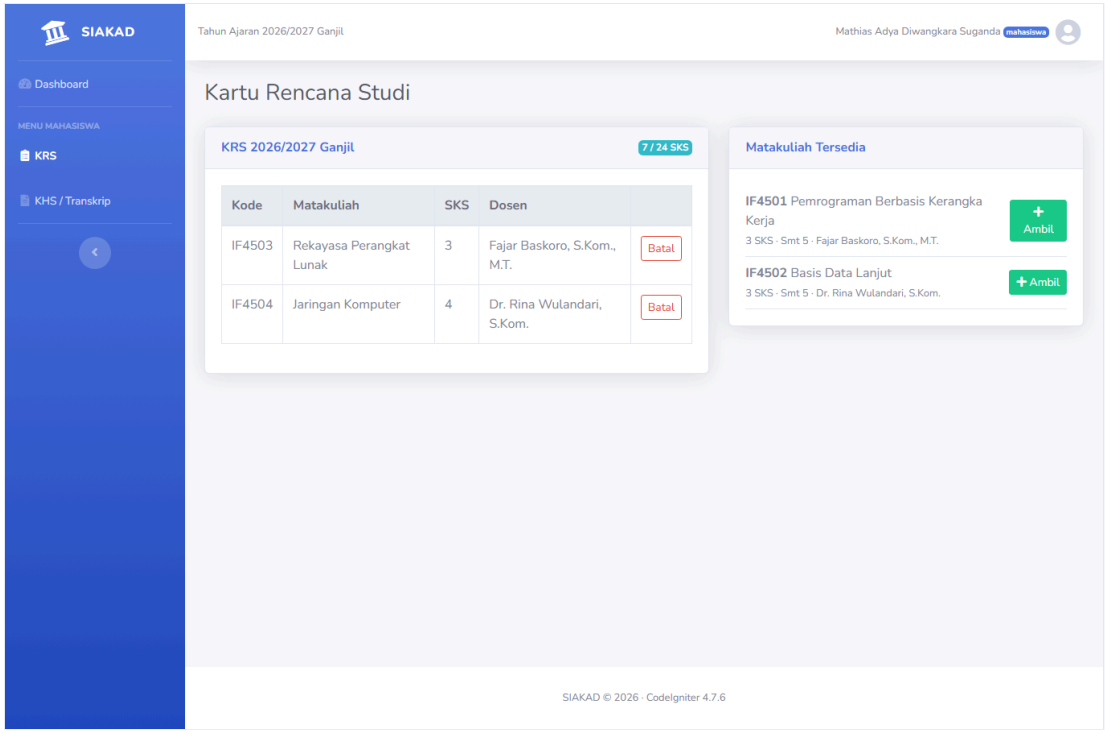
Gambar 8. Data matakuliah



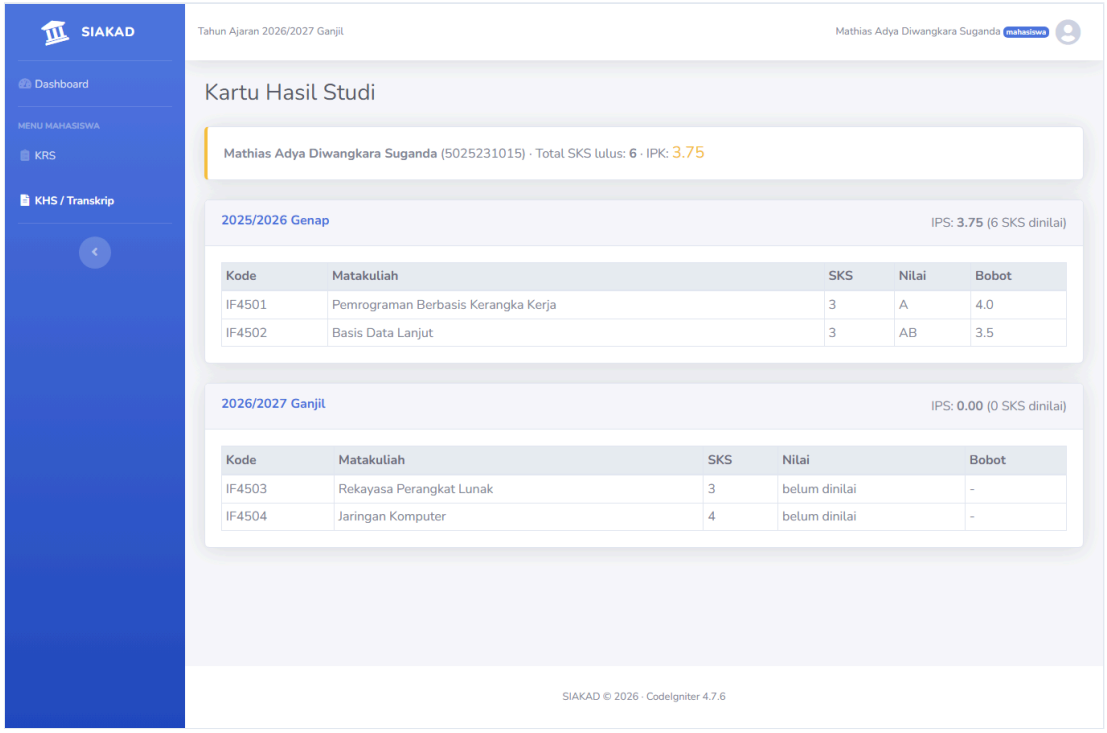
Gambar 9. Matakuliah yang sudah ada di KRS tidak bisa dihapus (FK RESTRICT)



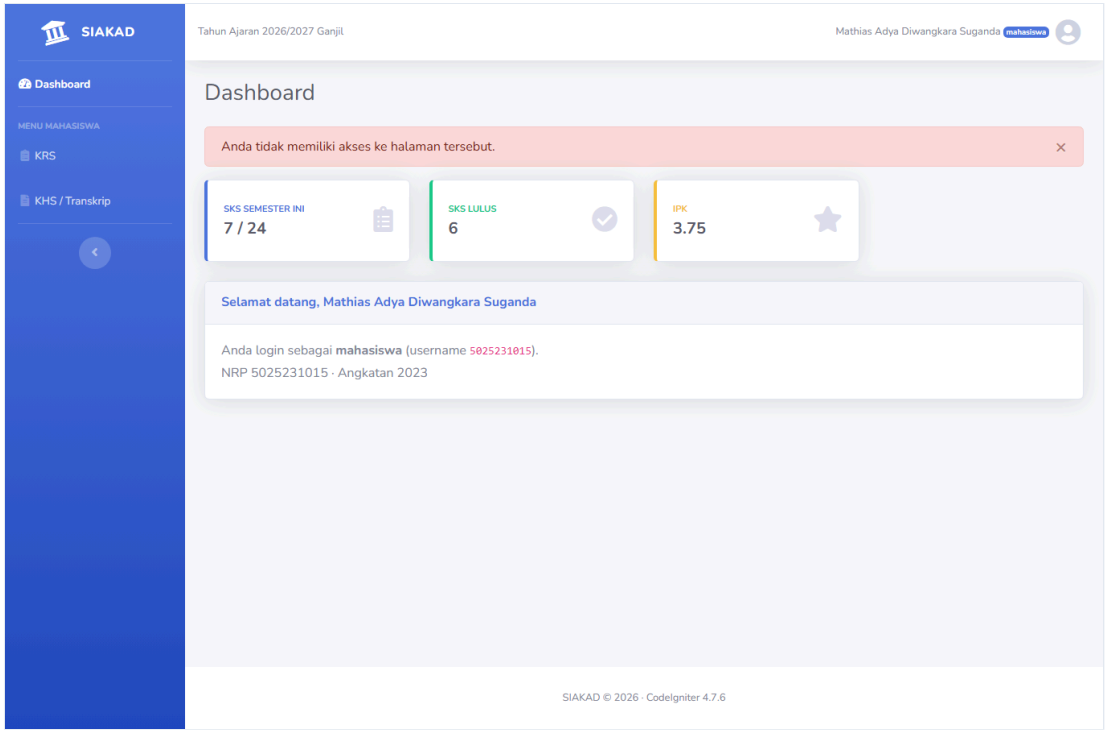
Gambar 10. Dashboard mahasiswa



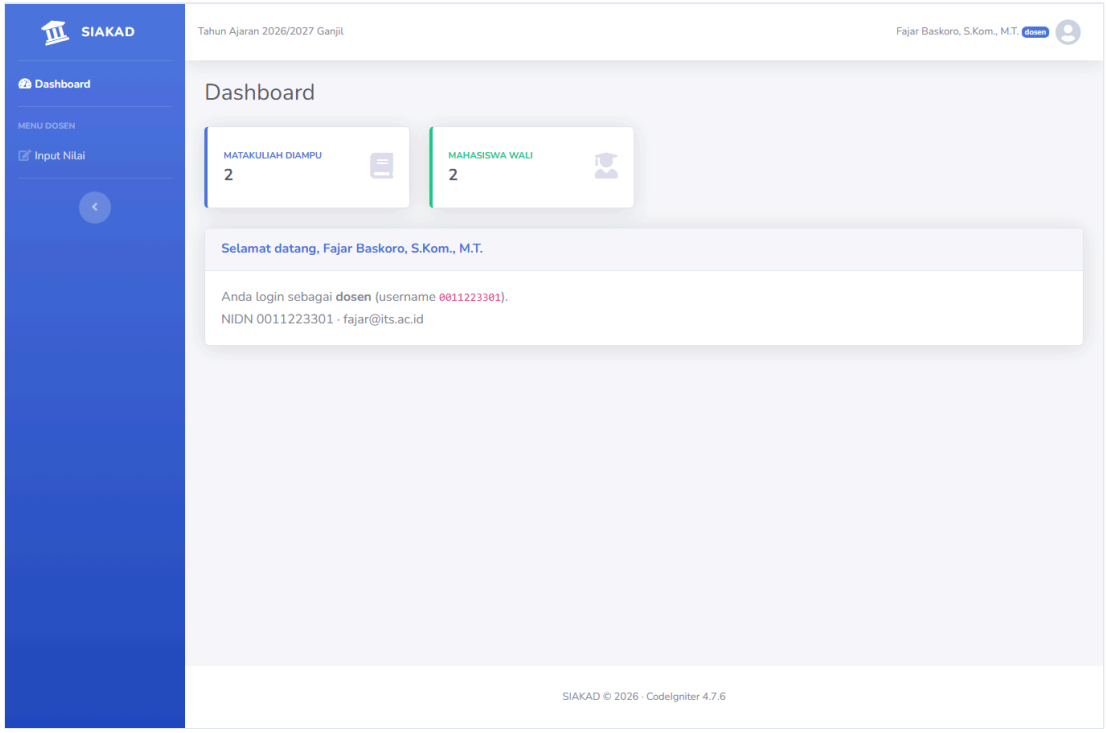
Gambar 11. Pengisian KRS



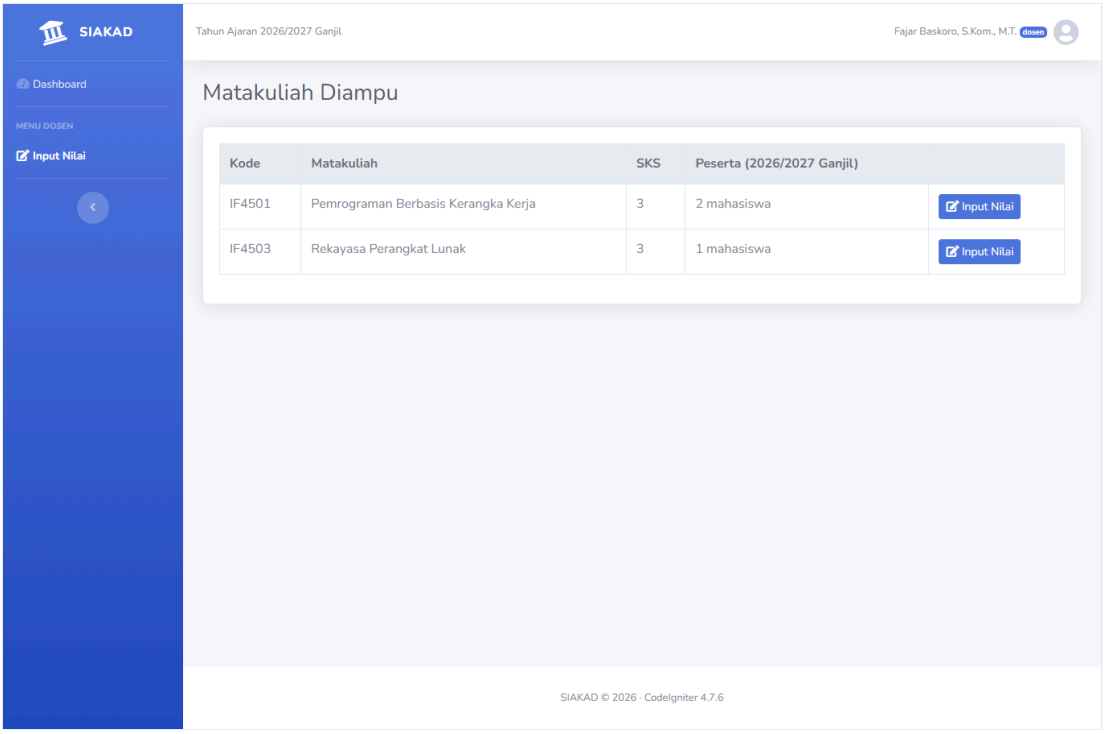
Gambar 12. KHS dengan IP dan IPK



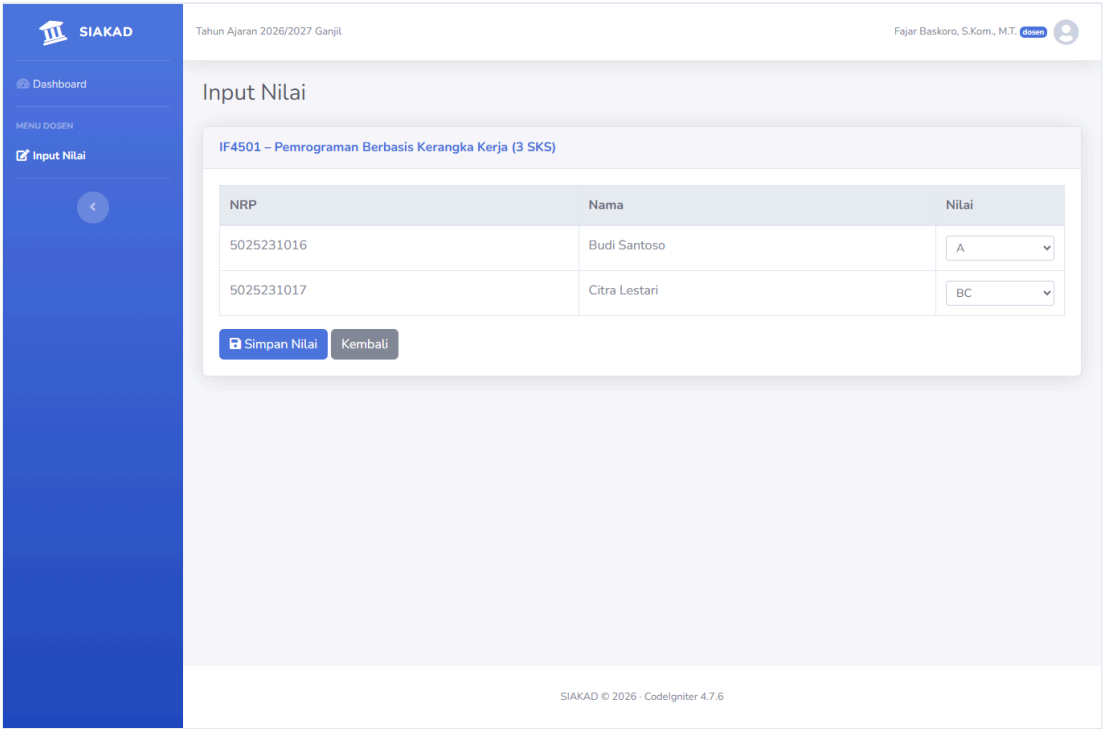
Gambar 13. Mahasiswa membuka halaman admin, akses ditolak



Gambar 14. Dashboard dosen

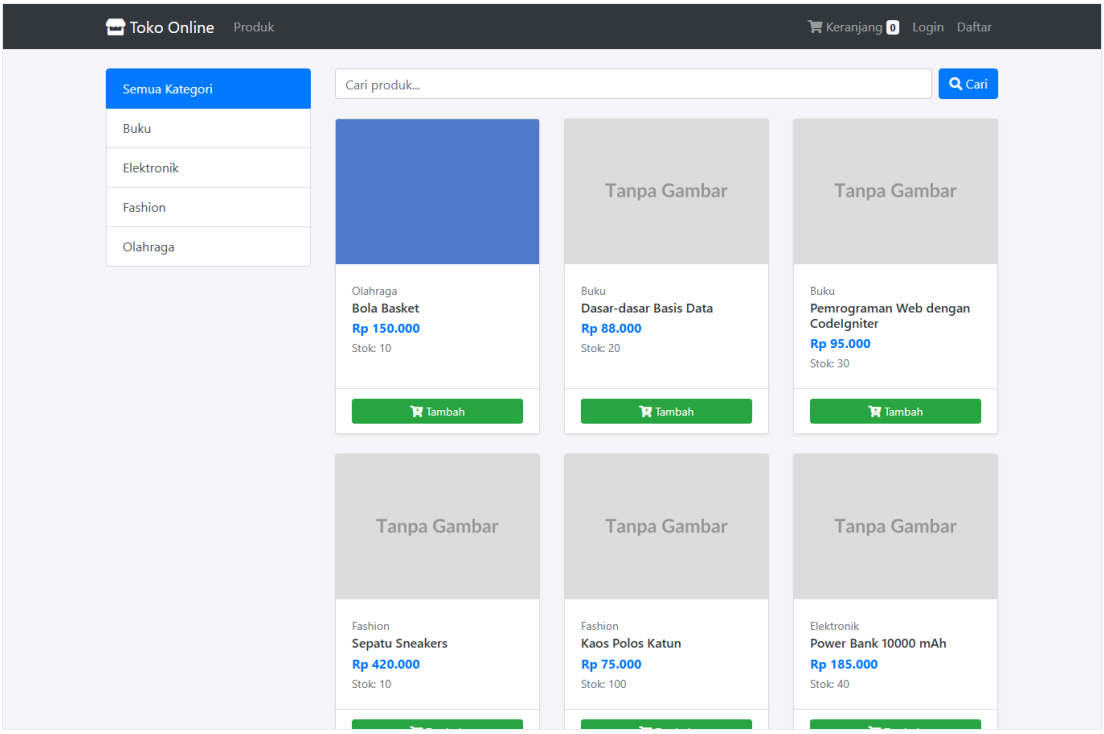


Gambar 15. Daftar matakuliah yang diampu dosen

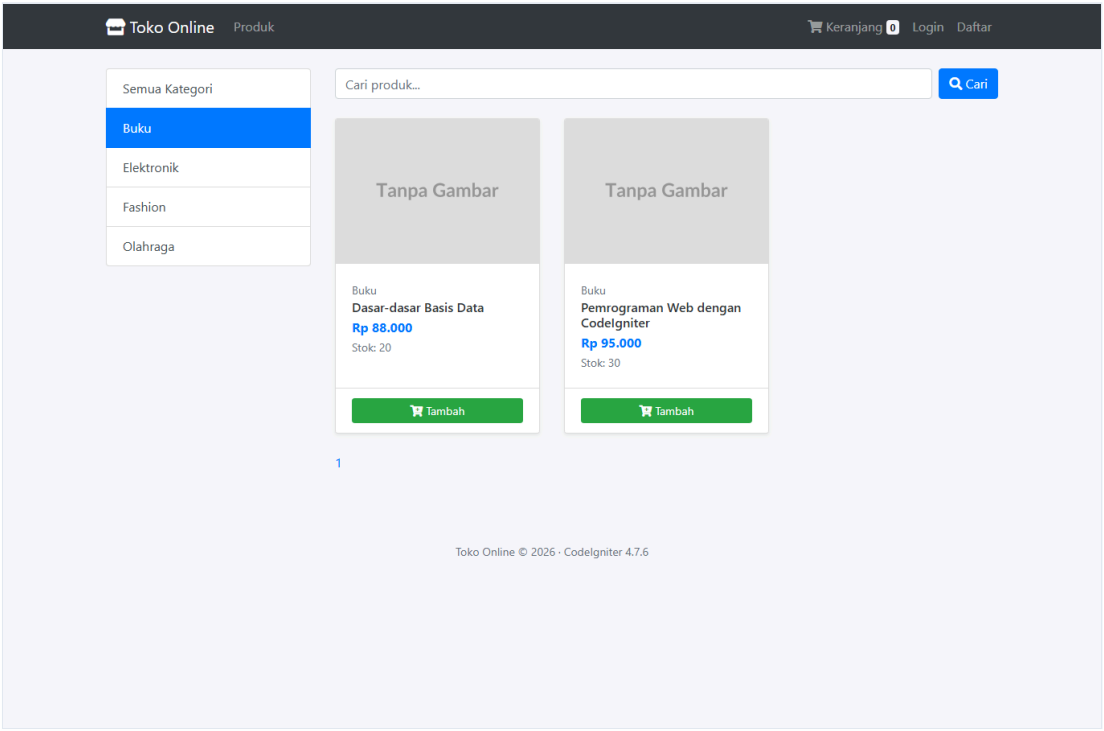


Gambar 16. Input nilai peserta

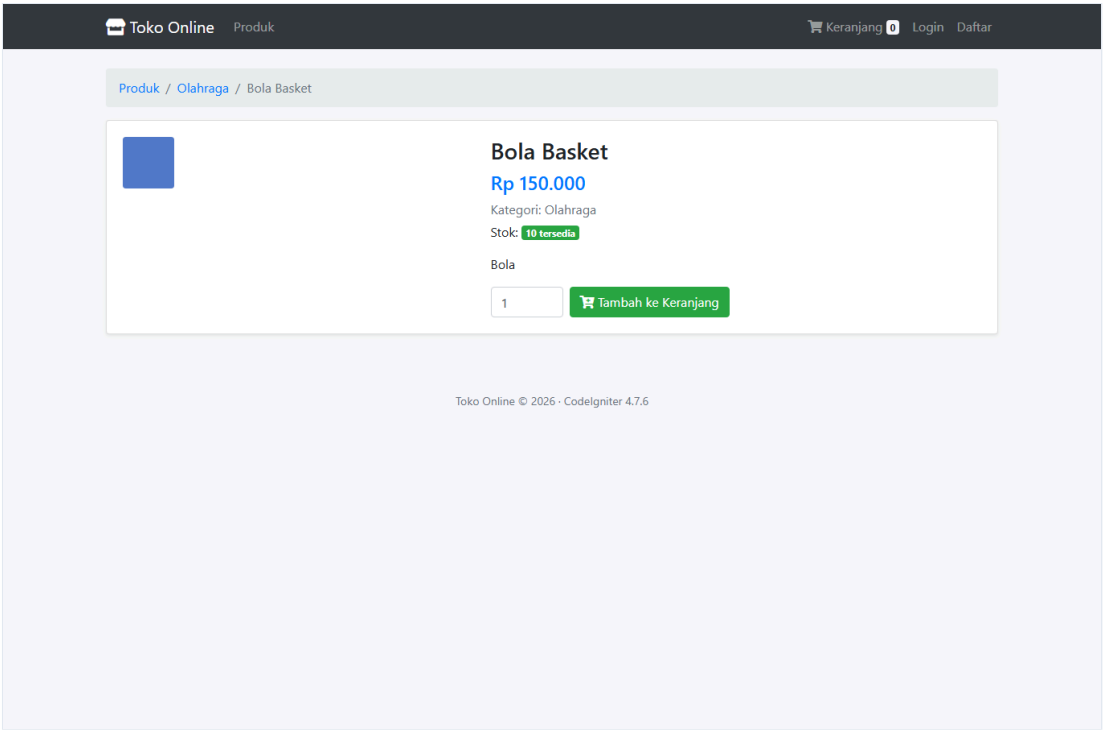
Toko Online



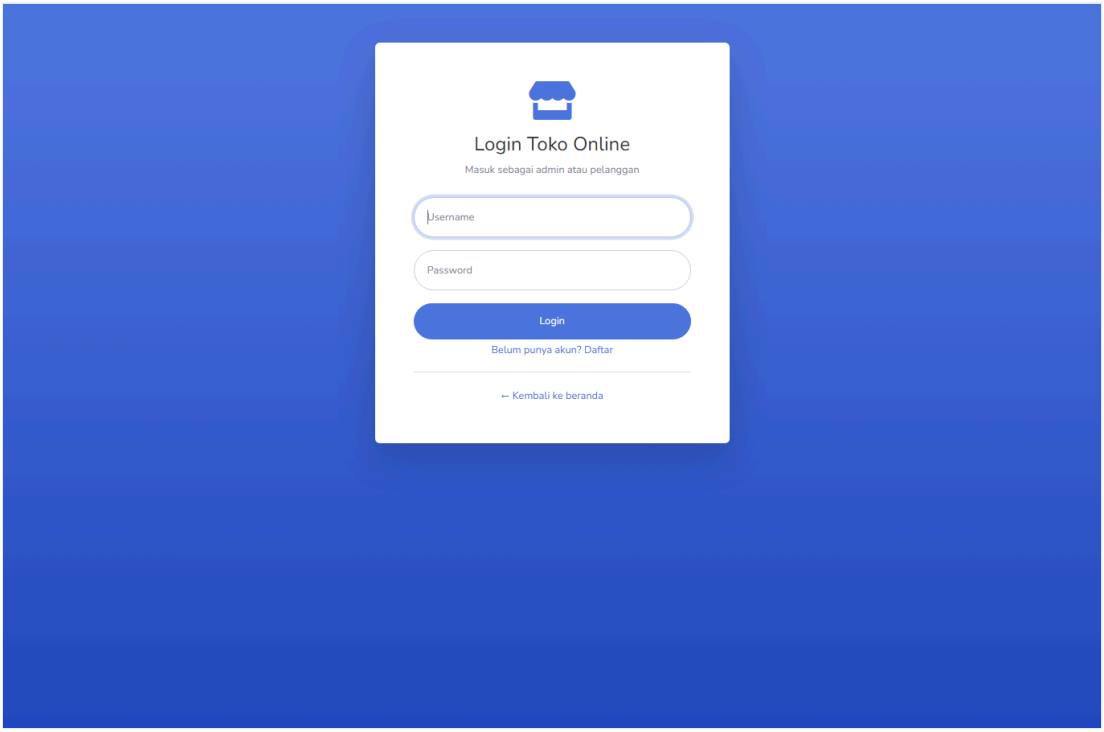
Gambar 17. Katalog produk (beranda)



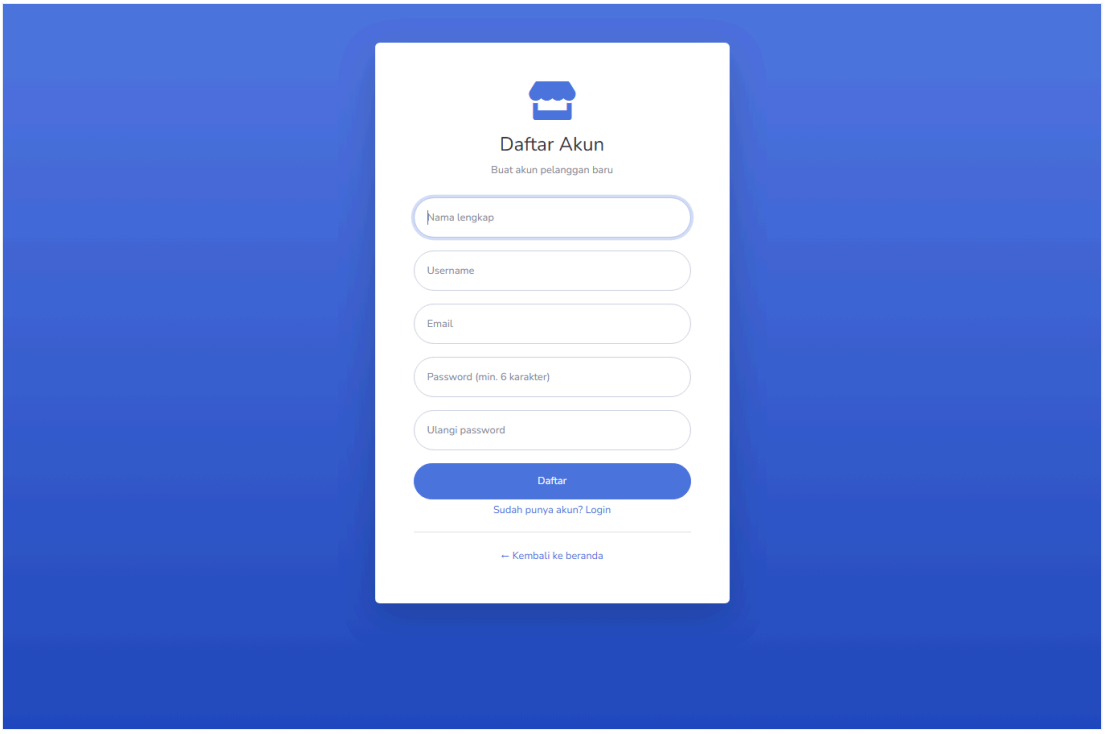
Gambar 18. Filter produk per kategori



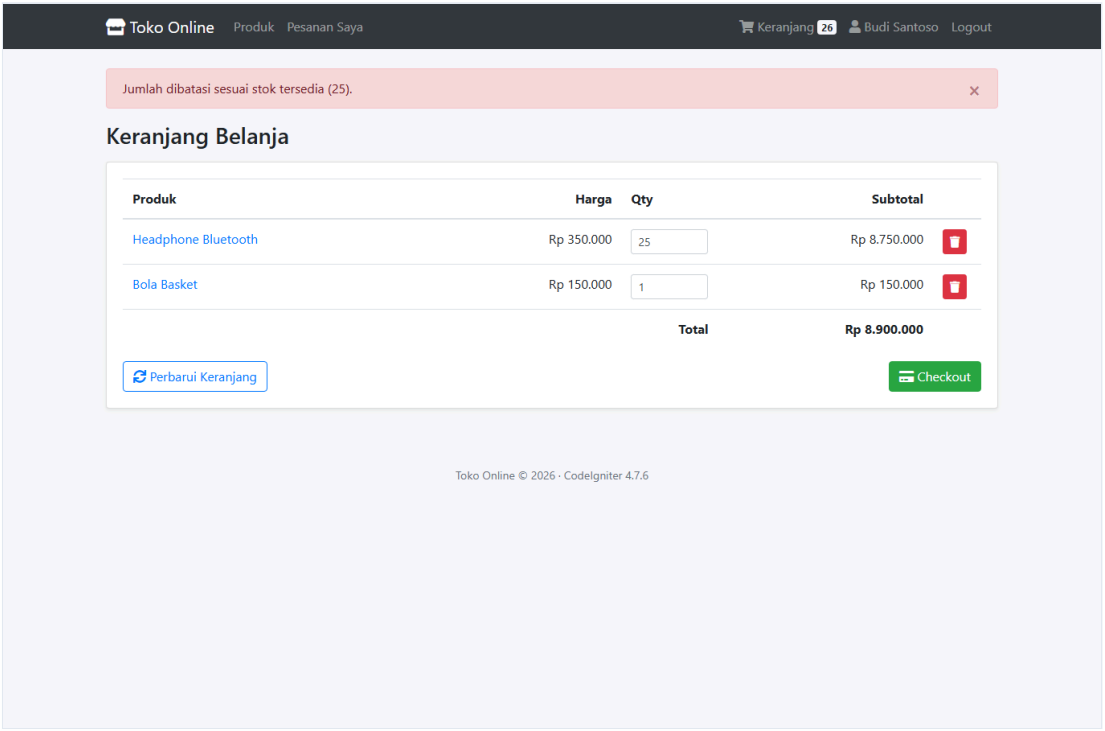
Gambar 19. Detail produk dengan gambar hasil upload



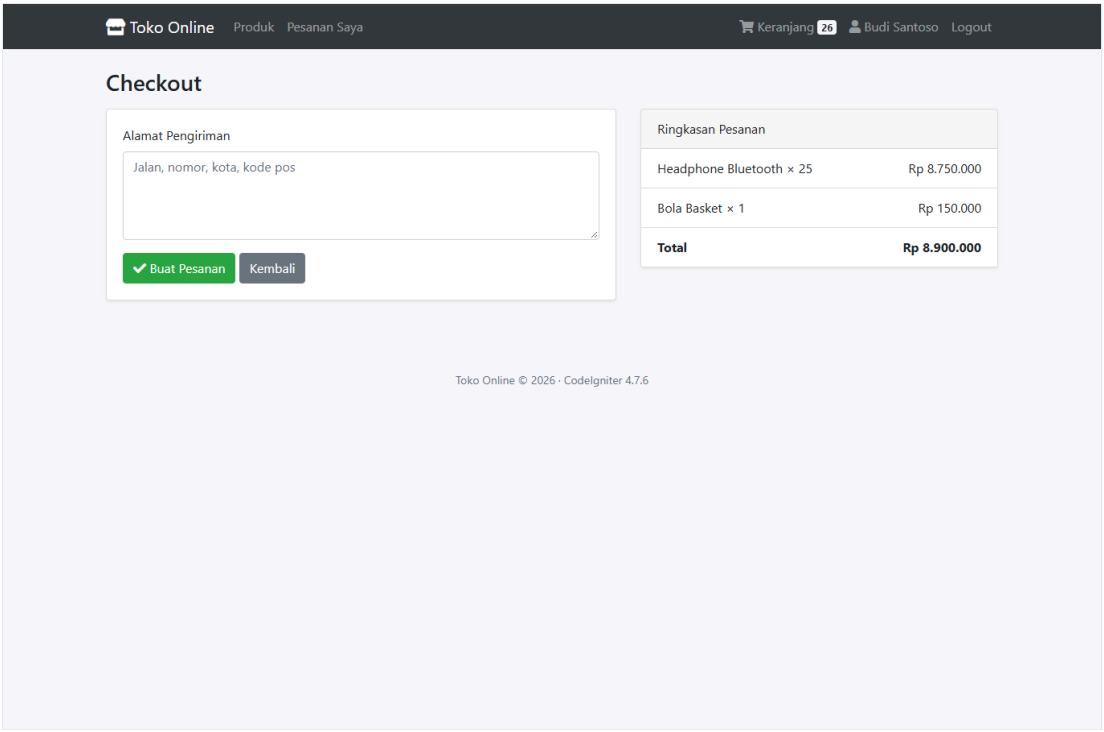
Gambar 20. Halaman login



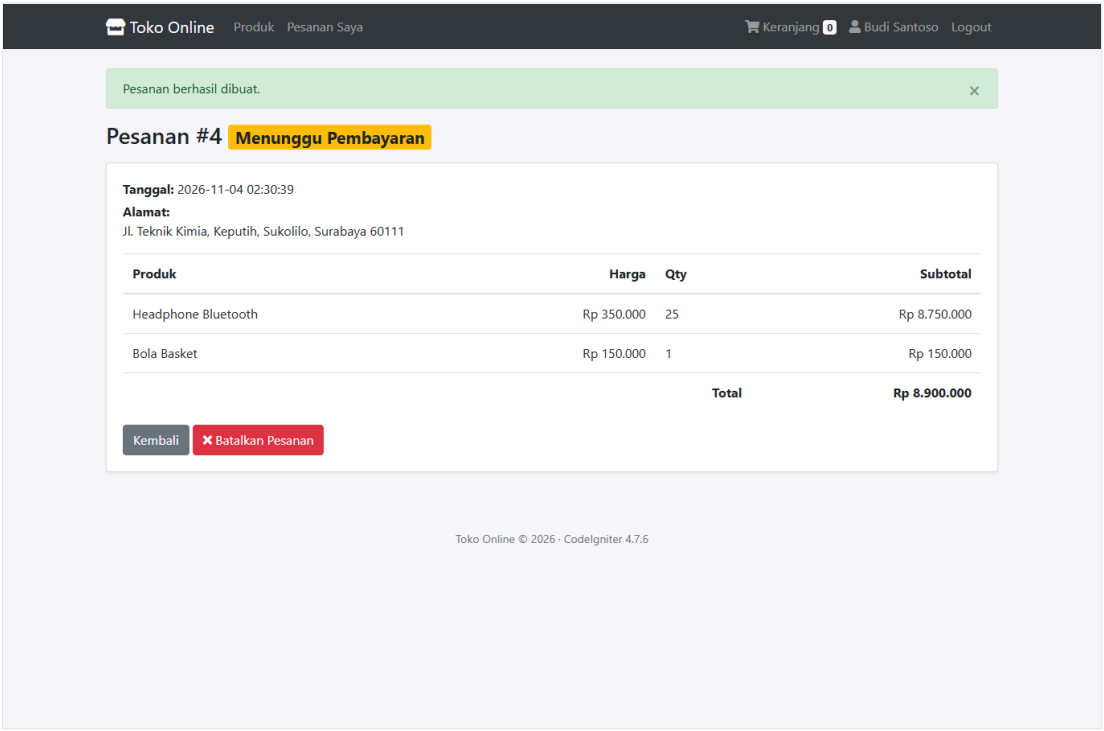
Gambar 21. Halaman registrasi pelanggan



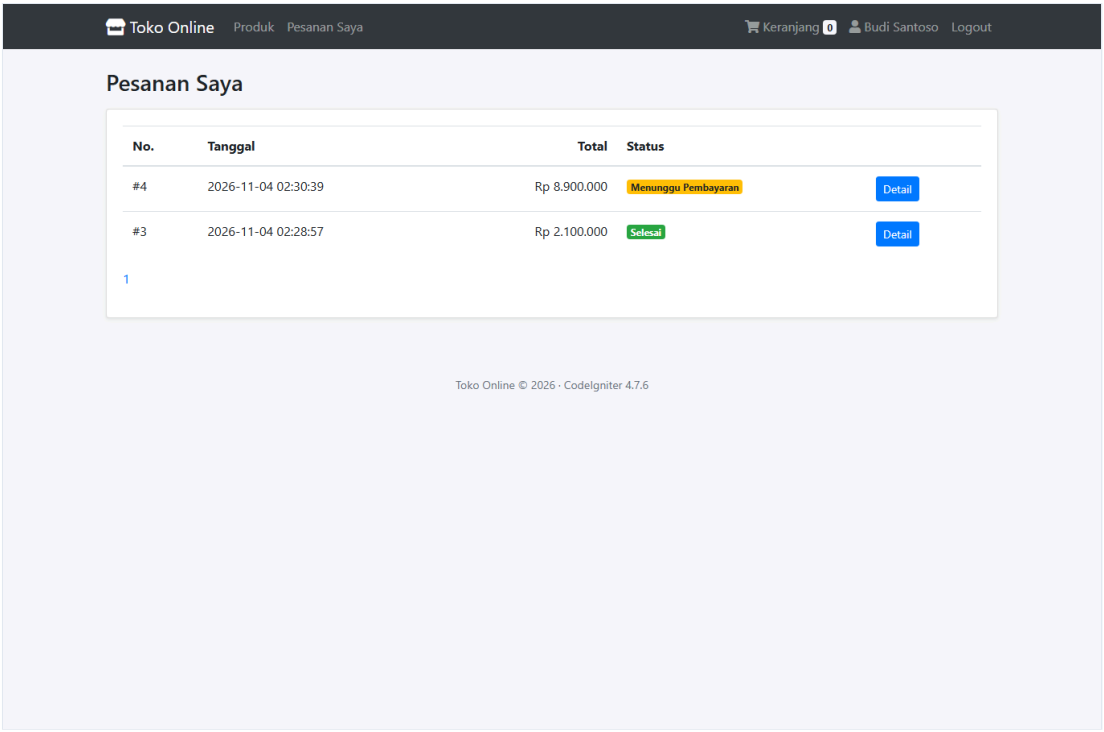
Gambar 22. Keranjang: jumlah dibatasi sesuai stok



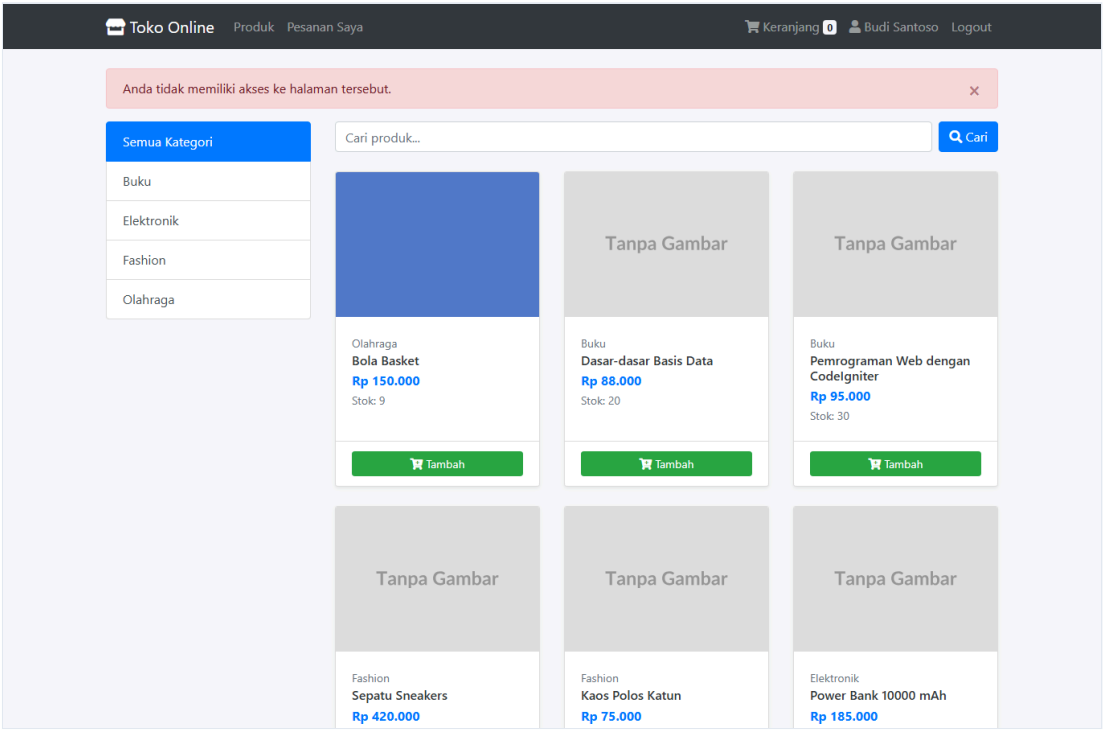
Gambar 23. Halaman checkout



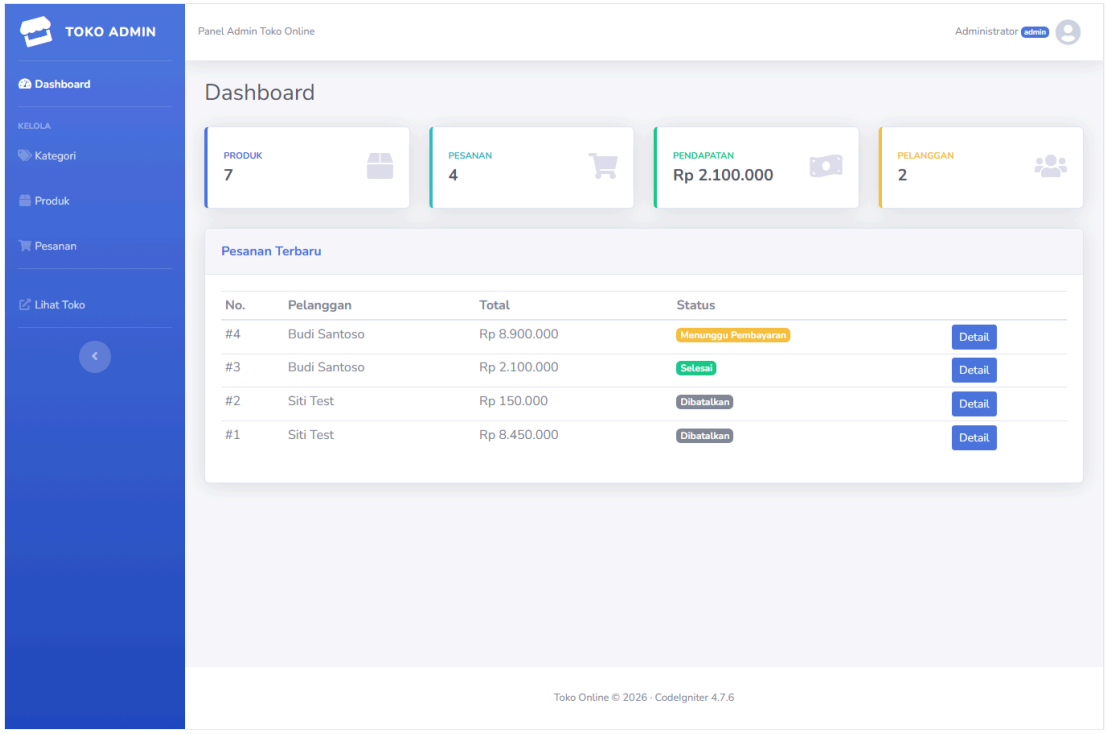
Gambar 24. Checkout berhasil, pesanan dibuat



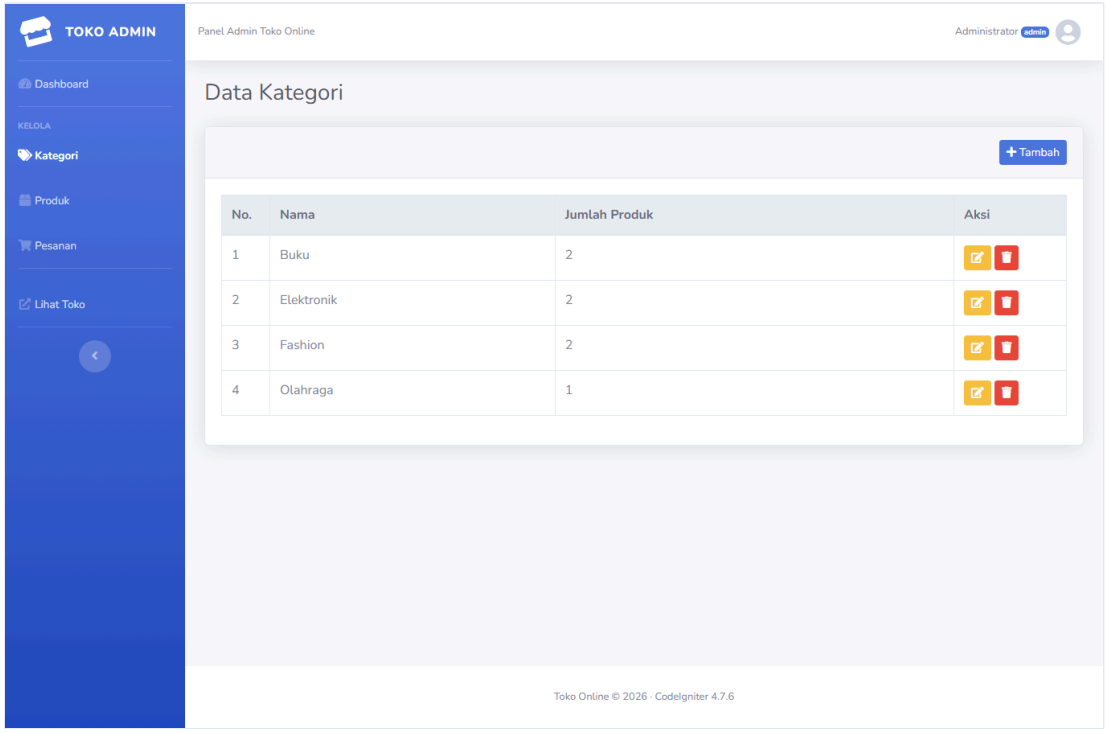
Gambar 25. Riwayat pesanan pelanggan



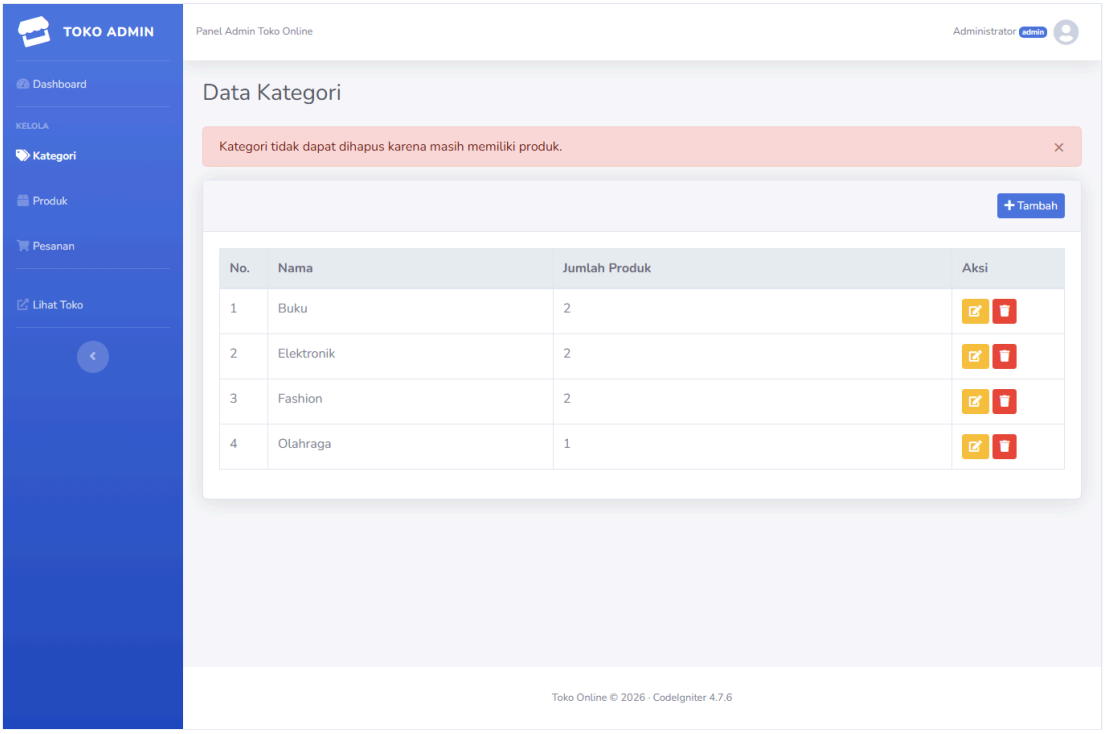
Gambar 26. Pelanggan membuka halaman admin, akses ditolak



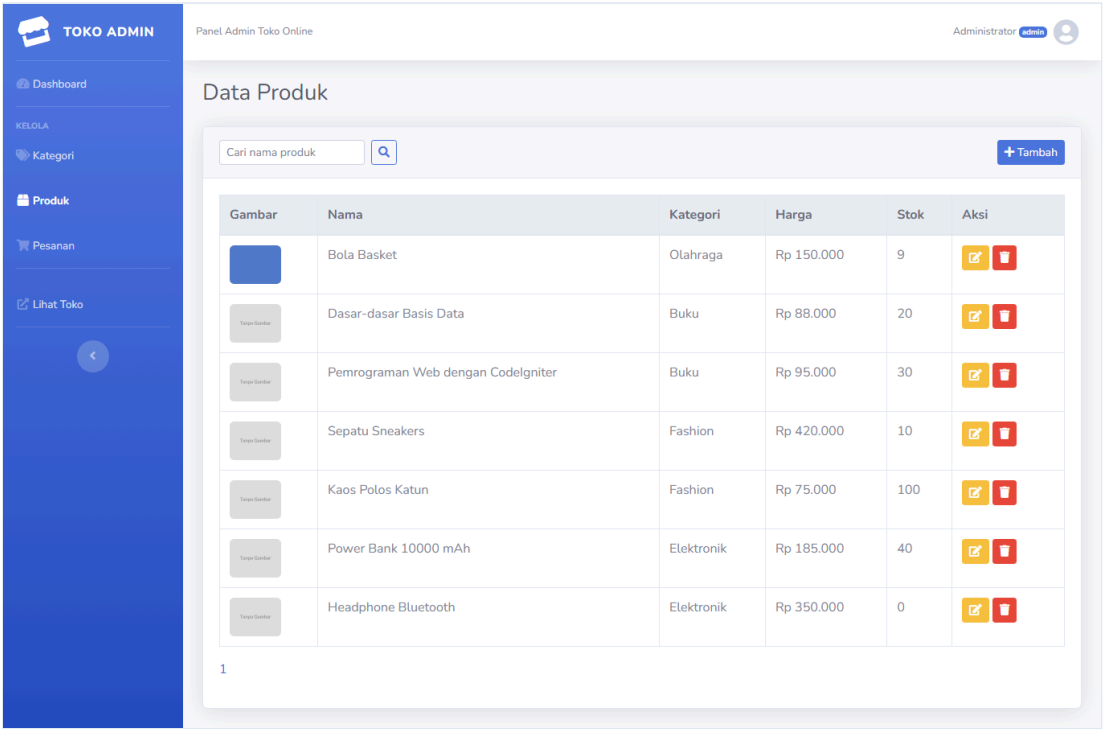
Gambar 27. Dashboard admin



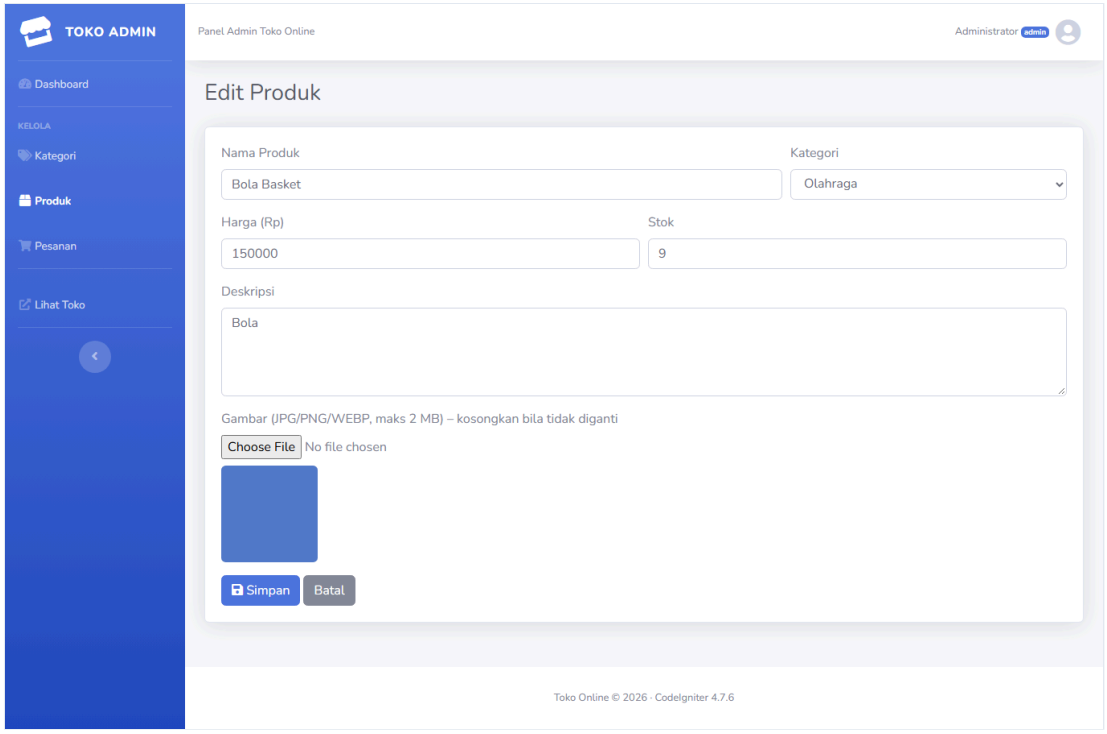
Gambar 28. Data kategori



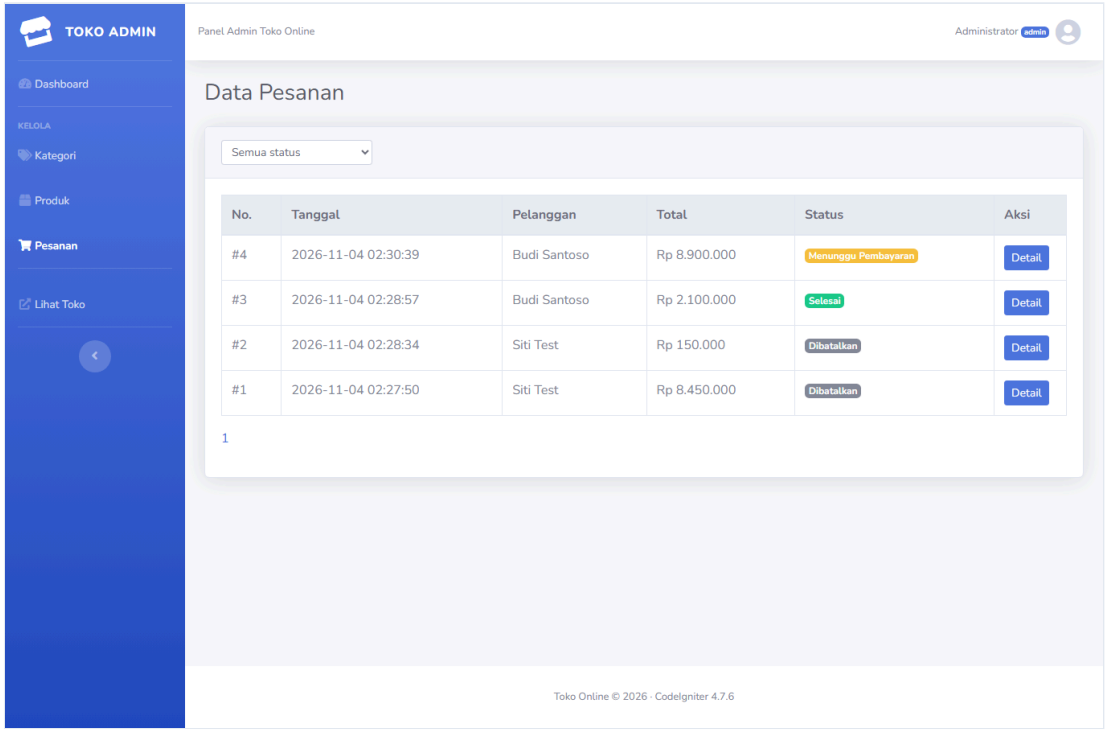
Gambar 29. Kategori yang masih memiliki produk tidak bisa dihapus (FK RESTRICT)



Gambar 30. Data produk



Gambar 31. Form produk dengan upload gambar



Gambar 32. Data pesanan untuk admin